Building Industrial RAG Systems from Daily Drilling Reports

A hands-on guide to retrieval-augmented engineering intelligence

Stephane Djimra

July 2026

Table of contents

Welcome

Every rig produces a paper trail. Every well produces a story — told in Daily Drilling Reports (DDRs), scattered across shared drives, scanned PDFs, and email attachments, written by dozens of different hands in a shorthand that only the field understands.

That story holds answers your team needs today: *Have we seen this stuck pipe signature before? What usually precedes a screenout on this formation? Which offset wells lost circulation at this depth?*

Somewhere in your archive, the answer already exists. Finding it is the problem.

This book builds, from scratch and in plain Python, a system that can read that archive and answer those questions — with evidence, not guesses. You will not start with theory. You will start with a single PDF and a script that reads it. By the end, you will have an Industrial DDR Intelligence Platform: a retrieval-augmented generation (RAG) system purpose-built for drilling and completions data, that you built chapter by chapter and understand end to end.

Every example in this book uses real Daily Drilling Reports — not invented ones. The archive is from **Utah FORGE**, a Department of Energy-funded geothermal research well (FORGE 16A(78)-32) whose reports are publicly available. No anonymisation, no synthetic stand-ins: you'll read genuine rig-floor language, including a real stuck-pipe event and a real fishing operation, from page one.

Who this book is for

You should read this book if you are a drilling engineer, completions engineer, intervention engineer, production engineer, petroleum data scientist, or part of a digital transformation team in energy, and you:

- Have never written a line of Python in your life — or tried once and gave up.
- Have strong operational experience and are comfortable in Excel.
- Have never built a RAG system, used a vector database, or worked with embeddings.
- Are curious about AI and automation, and are willing to type code examples yourself rather than just read about them.
- Want working tools, not a machine learning course.

No prior programming or AI/ML background is assumed. Every concept — even one as basic as what a function is — is introduced only after you've hit the problem it solves, never before. The goal of this book isn't to turn you into a software engineer. It's to make you capable of building useful engineering tools.

What you will build

Chapter	You will be able to…
1	Extract text from a DDR PDF
2	Expand oilfield abbreviations (BHA, WOB, MWD…) automatically
3	Keyword-search your entire DDR archive
4	Search DDRs by meaning, not just exact words
5	Ask a question and get a cited, evidence-backed answer
6–12	Scale the system to scanned reports, large archives, hybrid retrieval, reranking, and full traceability

How this book works

Every chapter solves one operational problem and ends with something you can run. There is no chapter you read passively — you will type code, run it against real DDR text, and see it work. Theory appears only when it explains *why* the code in front of you behaves the way it does.

We follow one rule throughout: **engineering usefulness over AI novelty**. Retrieval matters more than generation. Traceability matters more than eloquent prose. Engineers trust evidence, not predictions — so this system is built to show its work, every time.

Companion repository

All code, sample datasets, and notebooks referenced in this book live in the companion repository. See Appendix A for setup instructions. Every chapter lists exactly which files you need under **Repository files**.

Let's read our first DDR.

Preface

Why this book exists

I wrote this book because I kept seeing the same gap on drilling and completions teams: the field generates enormous amounts of written operational knowledge — Daily Drilling Reports, end-of-well reports, morning reports, NPT logs — and almost none of it is *usable* by the next engineer who needs it (Society of Petroleum Engineers 2022). It sits in PDF form, scanned or digital, named inconsistently, scattered across SharePoint folders and email threads. When a rig hits a screenout at 9,200 ft, the fastest way to find out if this has happened before is usually to ask the oldest person in the room.

Large language models and retrieval-augmented generation (RAG) are genuinely useful here — not because they are new or exciting, but because they solve a real, boring, expensive problem: turning an unstructured archive into something you can query. This book teaches you to build that system yourself, understand every part of it, and trust its output, rather than adopting a black-box product you cannot inspect.

Why "industrial"

A demo that answers one question about one PDF is easy to build and easy to find online. An industrial system is different. It has to:

- Handle scanned reports as well as digital ones.
- Scale to thousands of DDRs across dozens of wells.
- Never silently invent an answer — every claim must trace back to a specific report and page.
- Be auditable by someone who was not in the room when it was built.

Every chapter in Part II exists because a toy RAG system fails at one of these requirements. We fix them one at a time, in the order you would actually hit them in the field.

How to read this book

This book borrows the practical-first, project-based spirit of *Python Crash Course* (Matthes 2023) — small wins early, theory only when the code in front of you needs it, a working artifact at the end of every chapter — but assumes considerably less going in. You do not need to have written Python before. You do not need to know what a function or a loop is before Chapter 1; each concept is explained in plain language, translated into terms an engineer already knows, exactly when you need it to solve the problem in front of you.

Read it in order the first time. Each chapter assumes the code from the previous one exists and works — this is a build, not a reference manual. After your first pass, use it as a reference: jump to the chapter that matches the problem you currently have (Chapter 6 if your PDFs are scanned, Chapter 9 if your retrieval quality is poor, Chapter 10 if your answers aren't trustworthy yet).

Type the code yourself. Do not copy-paste from the companion repository until you have typed it once and watched it run. This is slower and it is worth it — and if you finish this book able to say "I can build this," rather than "I understand the theory," it did its job.

A note on data

Most Daily Drilling Reports are operationally sensitive and confidential — which is exactly why this book doesn't use one from a commercial operator. Every example here comes from **Utah FORGE**, a Department of Energy-funded enhanced geothermal system research well (FORGE 16A(78)-32) whose reports are public: real well name, real dates, real personnel, real events, exactly as filed. No anonymisation was applied, because none was needed — this is what "publicly available" means in practice, and it lets this book show you genuine field language instead of an approximation of it.

That said, the *technique* generalises directly to a confidential archive: everything from Chapter 6 onward — OCR gating, traceability, corpus-completeness checks — exists precisely because production archives are messier and more sensitive than this one. If you point this book's code at your own organisation's DDRs, do not commit them to a public repository, and see Appendix A for a data-handling checklist.

Acknowledgements

This book exists because of the drilling and completions engineers who patiently explained, over many years and many wells, why the report says “POOH to change BHA due to erratic torque” and what that sentence actually means for the next twelve hours of operations. Domain knowledge like that does not live in a textbook — it lives in people, and in the daily reports they write under time pressure at the end of a twelve-hour tour.

Thanks are due to the open-source communities behind the tools this book relies on throughout: Python, Quarto, Jupyter, pandas, and the broader Python data and NLP ecosystem. None of the engineering in this book would be reproducible without them.

Finally, thanks to every reader working through this material on a real archive of real reports. If you find an error, an oilfield abbreviation that’s wrong for your basin, or a chapter that assumes something it shouldn’t, please open an issue in the companion repository — corrections from practitioners are what keep a book like this honest.

Part I.

Part I — Foundations

1. Reading Your First DDR

```
PDF
 ↓
Text
```

1.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why DDR PDFs are hard for software to read, and the difference between a *digital-native* PDF and a *scanned* one.
- Extract raw text from a digital-native PDF using Python.
- Write a script that processes a single DDR, or an entire folder of them, from the command line.
- Save extracted text so it's ready for the cleaning work in Chapter 2.

1.2. Operational Problem

It's Monday morning. Someone on the call asks: *"Have we had a stuck pipe event on this well before? What happened, and how was it resolved?"* The answer is somewhere in the DDR archive — but that archive is 76 PDF files, named inconsistently by whatever software produced them, and the one report that matters is buried in there like a single page in a filing cabinet with no index. Nobody wants to open 76 PDFs one at a time to answer a question that took ten seconds to ask.

This book uses a real, publicly available archive to solve exactly that problem: the Daily Drilling Reports from **Utah FORGE** — a Department of Energy-funded research project at the University of Utah, drilling and testing an enhanced geothermal system (EGS) well, **FORGE 16A(78)-32**, in Beaver County, Utah. Because it's a public research well rather than a commercial operator's confidential asset, its DDRs are openly available — which means every example in this book is real, traceable text, not an invented stand-in.

Before we can search this archive, summarize it, or ask it questions, we need the words inside those PDFs available to a computer as plain text. That's the whole job of this chapter: given one PDF, get its text into something Python can work with. Nothing clever yet — no search, no abbreviation handling, no AI. Just a reliable way to turn a PDF into text. Everything in this book is built on top of getting this first step right.

1.3. Example DDR extract

Real DDR excerpt — report #38, FORGE 16A(78)-32, 2020-11-26

```
RPT DATE:11/26/2020
DAILY DRILLING REPORT RPT NUM.:38
WELL NAME:FORGE 16A [78]-32 JOB:Drilling 65° Tangent
PRESENT OPERATIONS:PIPE FREE, TRIP OUT OF HOLE FOR BHA INSPECTION

TIME BREAKDOWN
23:30 04:00 4.5 Drill From 6,360' to 6,507', (147') Total,
32.6' feet per hour.
WOB 20 TO 35k, Rotary 50, Torque 6,500, SPP 3200-3400 GPM 560,
DIFF 200-300psi
During the slide lost tool face and became assembly became stuck
04:00 06:00 2.0 Work pipe, circulate lube sweep, work tool back in position
Pipe free
```

This is one of ten real reports curated for Part I, spanning the well's actual timeline from rig-up in October 2020 through this stuck-pipe day in November and a fishing operation in December. Every excerpt in Part I is genuine, unedited report text — no anonymisation was needed, because this data is already public. Notice that report #38 answers the morning call question directly — *if* you happen to open this one file out of 76. That's the gap this chapter closes.

1.4. Theory

A PDF is not a text file wearing a costume. It's a page-description format: instructions for *drawing* characters at specific coordinates, not a stream of text with a beginning and an end. Two DDRs that look identical on screen can be built completely differently under the hood:

- **Digital-native PDFs** were generated directly from software (in this case, WellEz, the drilling data system that produced every report in this archive). The text was placed on the page as actual character data, so a library can recover it.
- **Scanned PDFs** are photographs of paper reports. There is no character data at all — just a picture of a page. Extracting text from these requires optical character recognition (OCR), which we cover in Chapter 6.

The Utah FORGE archive is entirely digital-native — every report was produced by the same software, so extraction is reliable throughout Part I. This won't always be true of a real archive (Chapter 6 deals with the scanned-report case), but it's the right place to start.

We'll use `pdfplumber`, a Python library that reads a PDF's internal structure and hands back the text (and, later in the book, tables and layout) on each page. There are other libraries that do this — `PyPDF2`, `pymupdf` — and they're worth knowing about, but `pdfplumber` is a good default: actively maintained, and its text extraction is reliable on the kind of structured reports we're working with.

> Engineering Translation: Library
>
> A **library** is borrowed tooling someone else already built and tested — the code equivalent of using a manufacturer's torque chart instead of deriving it yourself. `pdfplumber` is a library that already knows how to open a PDF and read the characters on its pages, so you don't have to write that from scratch.

```
DDR PDF
   ↓
open with pdfplumber
   ↓
for each page:
   ↓
  page.extract_text()  -->  string, or None if unextractable
   ↓
join every page's text together
   ↓
one Python string, ready for Chapter 2
```

1.5. Implementation

1.5.1. Step 1: get the sample archive

1.6. What problem are we solving?

Before extracting anything, confirm the report archive is actually there and see what you're working with — the same way you'd check a folder of scanned tickets exists before writing a script against it.

1.7. Inputs

- The folder `datasets/sample_ddrs/`, containing ten curated Utah FORGE DDR PDFs (already included in this repository — no download needed).

1.8. Expected Output

A list of ten file paths, sorted, something like:

```
FORGE-16A-78-32_Drilling_003_2020-10-22.pdf
FORGE-16A-78-32_Drilling_038_2020-11-26.pdf
...
FORGE-16A-78-32_Drilling_050_2020-12-08.pdf
```

```python
import pathlib
sorted(pathlib.Path("datasets/sample_ddrs").glob("*.pdf"))
```

1.9. What just happened?

You pointed Python at a folder and asked it to list every file ending in `.pdf`. Nothing was opened or read yet — this is just confirming the folder and its contents exist before you build anything on top of them.

> Engineering Translation: Path and glob
>
> A `Path` is Python's way of pointing at a file or folder location — think of it as a shortcut or bookmark to somewhere on disk. `glob("*.pdf")` is a filter: "show me everything in this folder whose name ends in `.pdf`," the same way you might filter a file browser to show only PDFs.

If you want to build this curated subset yourself from a full local copy of the public archive (or extend it with more reports), see `code/chapter_01/build_sample_archive.py` — it's documented in Appendix A.

1.9.1. Step 2: extract text from one PDF

1.10. What problem are we solving?

Prove that we can pull real text out of a single PDF before building anything more elaborate — the smallest possible version of the whole chapter's job.

1.11. Inputs

- One PDF: `datasets/sample_ddrs/FORGE-16A-78-32_Drilling_038_2020-11-26.pdf` (report #38, the stuck-pipe day).

1.12. Expected Output

The full text of the report's first page, printed to your terminal — header block, casing and mud tables, BHA component list, survey data, and the time breakdown quoted above, all as plain text.

```python
import pdfplumber

with pdfplumber.open(
    "datasets/sample_ddrs/FORGE-16A-78-32_Drilling_038_2020-11-26.pdf") as pdf:
    first_page = pdf.pages[0]
    print(first_page.extract_text())
```

1.13. What just happened?

You opened the PDF, grabbed its first page, and asked `pdfplumber` to hand back everything readable on it as plain text. Real DDRs pack in far more than a simple narrative — this one report alone has seven distinct data sections (header, casing, mud, BHA, survey, time breakdown, and remarks) — and all of it came back as one ordinary block of text you can now search, print, or save.

> Engineering Translation: Opening a file safely
>
> `with pdfplumber.open(...) as pdf:` is like signing a piece of equipment out and back in automatically. Everything inside the indented block happens while the file is "checked out"; once the block ends, Python closes it for you — so you never forget to release the file, even if something goes wrong partway through.

Every DDR in this archive is a single page, but that won't be true of every report you'll ever encounter. Loop over every page and keep track of which page each chunk of text came from — you'll need that later for citations:

1.14. What problem are we solving?

Handle reports with more than one page, and remember which page each piece of text came from — information you'll need in later chapters to point back to the exact source of an answer.

1.15. Inputs

- Any DDR PDF path, single-page or multi-page.

1.16. Expected Output

One combined string containing every page's text, with a marker like `--- Page 1 ---` showing where each page starts.

```
from pathlib import Path
import pdfplumber

def extract_text(pdf_path: Path) -> str:
    pages_text = []
    with pdfplumber.open(pdf_path) as pdf:
        for page_number, page in enumerate(pdf.pages, start=1):
```

```
            text = page.extract_text() or ""
            pages_text.append(f"--- Page {page_number} ---\n{text}")
    return "\n\n".join(pages_text)
```

1.17. What just happened?

This packages the previous step into a reusable procedure: for every page in the PDF, pull its text, label it with a page number, and stitch all the labelled pages together into one string. Instead of writing this logic out every time, you now have one named tool — `extract_text` — you can hand any PDF path to.

> Engineering Translation: Function and loop
>
> A **function** is a standard operating procedure: a named set of steps (`extract_text`) you can hand a new input to and trust it will follow the same procedure every time, instead of re-writing the steps by hand for each report. A **loop** (`for page_number, page in enumerate(...)`) means "repeat this operation for every page in the report" — the same way an SOP says "repeat for each joint" instead of listing every joint by hand.

Note the `or ""`. If a page has no extractable text — a scanned page mixed into an otherwise digital report, for example — `extract_text()` returns `None`, and every downstream step in this book expects a string, not `None`. This is a safety check, the code equivalent of confirming a valve position before proceeding instead of assuming it: assume real data is messier than the happy path, and this pattern shows up throughout the book.

1.17.1. Step 3: make it a script you can run from the command line

1.18. What problem are we solving?

Turn this into a tool you can point at any file or folder from the terminal, without opening or editing the code each time.

1.19. Inputs

- A single PDF path, or a folder of PDFs plus a `--batch` flag and an `--out` destination folder.

1.20. Expected Output

Printed text to the terminal (single-file mode), or a folder of `.txt` files, one per PDF (batch mode).

```
# One file, printed to the terminal
python code/chapter_01/read_ddr.py
↪  datasets/sample_ddrs/FORGE-16A-78-32_Drilling_038_2020-11-26.pdf

# A whole folder, saved as .txt files
python code/chapter_01/read_ddr.py datasets/sample_ddrs/ --batch --out
↪  datasets/ddr_text
```

1.21. What just happened?

The full script, `code/chapter_01/read_ddr.py`, wraps the `extract_text` function in a command-line interface: flags like `--batch` and `--out` work like switches on a control panel, letting you choose "one file, printed" or "whole folder, saved" without touching the code itself.

Try the first command now. You should see the full report text, including the line: `During the slide lost tool face and became assembly became stuck` — real language from a real rig report, not a paraphrase.

1.22. Production Reality

This chapter assumes every DDR is a clean, digital-native PDF produced by the same software — true for this entire archive, and the easiest possible starting point. A real multi-well, multi-operator archive usually isn't that tidy. Expect:

- scanned reports with no character data at all (Chapter 6)
- rotated or skewed pages
- duplicate reports filed under two different names
- handwritten notes or annotations layered on top of a digital report
- reports that are simply missing for a given day

None of that shows up in Part I's ten curated reports, on purpose — you need a reliable extraction script before you can handle the messy cases. Chapter 6 comes back to this list once you have somewhere solid to stand.

1.23. Practical exercise

Try it yourself: Run `read_ddr.py` in batch mode against `datasets/sample_ddrs/`, saving the output to `datasets/ddr_text/`. Then open `FORGE-16A-78-32_Drilling_050_2020-12-08.txt` and find the line describing what the crew milled up that day.

You'll know it worked when: you have ten `.txt` files in `datasets/ddr_text/`, one per PDF, and you can find the word "Fishing" in report 050's text without opening the original PDF.

1.24. Field notes

⚠ Field notes: a table that reads like nonsense until you know why

Action: extract report #38's `BHA` section and read it as plain text.
Result:

```
# COMPONENT OD ID LENGTH # COMPONENT OD ID LENGTH
1 Drill Bit-Reed SKC613M-01C 8.75 1 7 Monel Collar-NMDC
with MWD 6.75 3.25 30.35
2 Motor-6.5'' 7/8, 5.7, 0.242 RPG 1.50,* Fixed 6.5 33.01
8 Monel Collar-NMDC (2 joints) 6.75 3.25 60.66
```

Why: the real BHA table on this report is laid out as **two side-by-side sub-tables** — components 1–6 on the left half of the page, components 7–10 on the right half. Visually, that's obvious: two neat columns. But `extract_text()` reads left to right, top to bottom, so component 1's row and component 7's row — which just happen to sit on the same horizontal line of the page — get concatenated into one line of output text, with no marker showing where the left sub-table ends and the right one begins.
Lesson: `extract_text()` gives you the words on the page, not the *layout* those words depend on to make sense. A two-column table isn't a special case — it's normal DDR formatting — and reading its flattened output at face value would have you believe component 1 and component 7 are the same row. This book's chapters work around it by favouring the narrative `TIME BREAKDOWN` text over the structured tables for search and retrieval — real table-aware parsing (detecting table boundaries and reconstructing rows correctly) is its own discipline, and the companion pipeline's `src/rag_pdf/table_detect.py` and `table_extract.py` handle it properly. For now, just know that "extracted text" and "correct reading order" are not the same guarantee.

1.25. Challenge exercise

Challenge: Extend the script to also print, for each PDF, the total number of pages and the total character count extracted. Then compare report #3 (October 22, before the well was spudded — `PRESENT OPERATIONS: RIGGING UP...`) against report #38 (the stuck-pipe day): how much richer is the later report's text, and why might that be? (Hint: check the `SPUD DATE` and `DFS/DOL` fields in each report's header.) A reference solution is in `code/chapter_01/challenge/`.

1.26. Key takeaways

- A PDF's internal structure, not its on-screen appearance, determines whether text can be extracted directly. Digital-native PDFs can; scanned PDFs need OCR (Chapter 6).
- `pdfplumber` turns a PDF into plain Python strings, one page at a time.
- Handle `None` from `extract_text()` explicitly — real-world DDR archives will contain pages that don't extract cleanly, and your code should never crash on them.

- A five-line function (`extract_text`) plus a thin command-line wrapper is already a genuinely useful tool. You don't need machine learning to save an engineer an hour of searching PDFs by hand — and it works identically whether the PDF is a synthetic example or, as here, a real report from a real well.

1.27. Repository files

File	Purpose
`code/chapter_01/read_ddr.py`	Extracts text from one PDF or a folder of PDFs
`code/chapter_01/build_sample_archive.py`	Reproduces the curated Part I subset from the full public archive
`datasets/sample_ddrs/`	Ten real, curated Utah FORGE DDR PDFs used throughout Part I
`notebooks/chapter_01_explore.ipynb`	Interactive notebook version of this chapter

1.28. What can you do now that you couldn't do before?

You can turn any digital-native DDR PDF — or a whole folder of them — into plain text you can read, search, or hand to another script, without opening a single PDF viewer.

1.29. Suggested next step

The text you just extracted uses real oilfield shorthand and drilling terminology — `BHA`, `WOB`, `SPP`, `P JSM` — some spelled out, some abbreviated, exactly as the field actually writes it. Chapter 2 builds an abbreviation expansion engine grounded in what's actually in this archive, turning this raw text into something both humans and machines can search reliably.

2. Cleaning Operational Text

```
Raw Text
 ↓
Expanded Text
```

2.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why oilfield shorthand breaks naive text search and NLP.
- Build a dictionary-based abbreviation expander that respects word boundaries so it doesn't corrupt unrelated text.
- Apply the expander across every `.txt` file produced in Chapter 1.

2.2. Operational Problem

Next chapter, you're going to build a search tool and ask it: *"Show me every report that mentions the bottom hole assembly."* Will it find report #38? Open the `.txt` file you extracted from it in Chapter 1 and check: the report never once writes "bottom hole assembly." It writes `BHA` — twenty-one times. A search tool that only knows the literal words you typed will come back empty, on a report that was exactly the one you needed.

2.3. Why keyword search fails

Question: *"Show me every report that mentions the bottom hole assembly."*

Keyword search for "bottom hole assembly" finds nothing in report #38, because the report never uses that phrase. Instead it says `BHA` — the same equipment, different word. A computer matching text literally has no way to know these mean the same thing. Report #38 also talks about `WOB`, `MWD`, `SPP`, and a dozen other abbreviations a keyword search would need to be told about, one by one, before it could ever find them.

Before we can search, summarise, or hand this text to a language model, we need to expand it into language that means the same thing to a human and to a machine. That's the whole job of this chapter.

2.4. Example DDR extract

Real excerpt — report #38, showing both patterns

```
PJSM, pre job safety meeting.                     <- self-expanded by source
...
Pick up Curve assembly 2, BHA 21, Reed Hycalog     <- BHA never expanded
SKC613M-01C Trip in hole to 5,200'
...
WOB 20 TO 35k, Rotary 50, Torque 6,500, SPP        <- WOB, SPP never expanded
3200-3400 GPM 560, DIFF 200-300psi
```

2.5. Theory

An abbreviation expander is just a dictionary lookup — `{"BHA": "bottom hole assembly", "WOB": "weight on bit", ...}` — but two details make or break it:

1. **Word boundaries.** `str.replace("MD", ...)` will happily mangle a word that merely *contains* "MD" as a substring. Match on whole words using regex `\b...\b`, not raw substring replacement.
2. **Case.** DDRs mix cases inconsistently — this archive's headers are often all-caps while narrative text is mixed-case — so expansion needs to be case-insensitive without forcing the rest of the sentence to change case.

Engineering Translation: Dictionary

A Python **dictionary** is an equipment lookup table: you give it a tag (`"BHA"`) and it gives you back what that tag means (`"bottom hole assembly"`) — the same way a lookup table on the rig floor maps a short code stamped on a piece of equipment to its full specification.

This is deliberately not a machine-learning problem. A few dozen well-chosen abbreviations, expanded correctly, solve most of the readability problem — and unlike a model, a dictionary is auditable: you can list every expansion it will ever make.

```
raw text
   ↓
for each (abbreviation, expansion) in dictionary:
   ↓
  build pattern: \b + abbreviation + \b   (case-insensitive)
   ↓
  substitute every match with the expansion
   ↓
expanded text
```

2.6. Implementation

2.6.1. Step 1: build the lookup table

2.7. What problem are we solving?

Give the computer the same knowledge a drilling engineer already carries in their head: what each abbreviation stands for.

2.8. Inputs

- A hand-built list of oilfield abbreviations and their full-text expansions, drawn from what actually appears in this archive.

2.9. Expected Output

A Python dictionary — no output printed yet, just a lookup table in memory, ready for the next step to use.

```python
# code/chapter_02/expand_abbreviations.py
ABBREVIATIONS = {
    "BHA": "bottom hole assembly",
    "WOB": "weight on bit",
    "RPM": "revolutions per minute",
    "ROP": "rate of penetration",
    "MD": "measured depth",
    "TVD": "true vertical depth",
    "SPP": "stand pipe pressure",
    "GPM": "gallons per minute",
    "DLS": "dogleg severity",
    "MWD": "measurement while drilling",
    "NMDC": "non-magnetic drill collar",
    "UBHO": "universal bottom hole orientation sub",
    "NPT": "non-productive time",
    "ECD": "equivalent circulating density",
    "MW": "mud weight",
    "DFS": "days from spud",
    "DOL": "days on location",
    "PDC": "polycrystalline diamond compact",
}
```

2.10. What just happened?

You wrote down, once, what every abbreviation in this archive means. This list is the entire "knowledge" the expander needs — no training, no model, just a table you can read top to bottom and check by eye.

2.10.1. Step 2: expand every abbreviation safely

2.11. What problem are we solving?

Turn `BHA` into `bottom hole assembly` everywhere it appears as its own word — without also mangling words that merely happen to contain those same letters.

2.12. Inputs

- Raw report text (a Python string).
- The `ABBREVIATIONS` lookup table from Step 1.

2.13. Expected Output

The same text, with every whole-word abbreviation replaced by its expansion — for example, `Pick up Curve assembly 2, BHA 21` becomes `Pick up Curve assembly 2, bottom hole assembly 21`.

```python
import re

def expand_text(text: str, abbreviations: dict[str, str] = ABBREVIATIONS) -> str:
    for abbr, expansion in abbreviations.items():
        pattern = r"\b" + re.escape(abbr) + r"\b"
        text = re.sub(pattern, expansion, text, flags=re.IGNORECASE)
    return text
```

2.14. What just happened?

For every entry in the lookup table, this checks the text for that exact word — not just those letters anywhere, but that whole word on its own — and swaps in the full expansion, regardless of whether it was typed in capitals or lowercase.

Engineering Translation: Word boundary matching

`\b...\b` is a stricter kind of find-and-replace: instead of "contains these letters anywhere," it means "matches this exact word, with a space or punctuation on either side." That's the difference between correctly expanding the word `MD` and accidentally mangling it inside an unrelated word that merely contains those two letters.

2.14.1. Step 3: run it across the whole archive

2.15. What problem are we solving?

Apply the expander to every report at once, not just one file at a time.

2.16. Inputs

- The folder of `.txt` files Chapter 1 produced: `datasets/ddr_text/`.

2.17. Expected Output

A new folder, `datasets/ddr_text_expanded/`, with one expanded `.txt` file per input file, same filenames.

```
python code/chapter_02/expand_abbreviations.py \
    --in-dir datasets/ddr_text --out-dir datasets/ddr_text_expanded
```

2.18. What just happened?

The script read every `.txt` file Chapter 1 produced, ran each one through `expand_text`, and saved the result under the same filename in a new folder — so you can always find the expanded version of a report by matching its name against the original.

2.19. Production Reality

This chapter's dictionary covers the abbreviations that actually appear in this archive — because it's all one well, produced by one operator's reporting software. A larger, multi-well or multi-operator archive is rarely that uniform. Expect:

- different operators abbreviating the same equipment differently
- typos and inconsistent spacing (`BHA`, `B.H.A.`, `B H A`)

- unit mismatches hiding behind the same abbreviation (`MW` as mud weight on one report, something else entirely on another)
- abbreviations this dictionary has simply never seen before

None of that appears in Utah FORGE's reports, which is exactly why this is the right place to learn the technique before facing a messier archive.

2.20. Practical exercise

Try it yourself: Run the expander against `FORGE-16A-78-32_Drilling_038_2020-11-26.txt`.

You'll know it worked when: `Pick up Curve assembly 2, BHA 21` becomes `Pick up Curve assembly 2, bottom hole assembly 21`, and no unrelated word (check the `MD/TVD` survey table carefully — short abbreviations are the ones most likely to collide with real words) gets corrupted.

2.21. Field notes

> ⚠ Field notes: the source already expands some abbreviations — but not the ones that matter
>
> **Action:** search every report for the literal string `PJSM`.
> **Result:** it appears in six of the ten sample reports (not logged on rig-up or trip days when no fresh crew safety meeting was held) — but every single time it does appear, it's immediately followed by its own expansion, written by the source software itself: `PJSM, pre job safety meeting`.
> **Why:** this archive's report-generation software (WellEz) evidently has a template that spells "pre job safety meeting" out in full every time, right after the abbreviation. But run the same check against `BHA`, `WOB`, `MWD`, or `SPP` — the terms that actually carry operational meaning in the time breakdown — and none of them ever get that treatment, anywhere in the archive.
> **Lesson:** don't assume a real archive is consistently either abbreviated or expanded. This one is both, unevenly, by accident of whatever template a report-writing tool happened to use for one specific phrase. An automated expander has to cover every term regardless — you can't rely on the source to have already done the job for the ones that matter.

2.22. Challenge exercise

Challenge: Extend `ABBREVIATIONS` using Appendix B, and handle the mud-table abbreviations (`FV`, `WL`, `PV`, `YP`, `CL`) — notice how short these are, and think carefully about whether blindly expanding a two-letter token like `PV` is actually safe against this archive's real text, or whether it needs a narrower match context. A reference solution is in `code/chapter_02/challenge/`.

2.23. Key takeaways

- Word-boundary regex, not substring replacement, is the difference between a correct expander and a silently broken one.
- Real archives are inconsistent by nature — the same report can spell one abbreviation out in full while never expanding another. Don't assume the source data will document itself.
- A transparent dictionary beats an opaque model here: every expansion is inspectable and testable.
- Cleaning text is not optional groundwork — it's what makes every later chapter's search and retrieval actually work.

2.24. Repository files

File	Purpose
`code/chapter_02/expand_abbreviations.py`	Word-boundary abbreviation expander
`datasets/ddr_text_expanded/`	Expanded output from Chapter 1's extracted text
`notebooks/chapter_02_explore.ipynb`	Interactive companion notebook

2.25. What can you do now that you couldn't do before?

You can take the raw text Chapter 1 extracted and turn every shorthand term in it into plain language — so a report that only ever says `BHA` can now be found by anyone (or anything) searching for "bottom hole assembly."

2.26. Suggested next step

With clean, expanded text in hand, Chapter 3 builds the first tool an engineer would actually reach for: a keyword search engine that answers "which reports mention losses?" in milliseconds.

3. Searching DDRs

```
Text
 ↓
Searchable Archive
```

3.1. Learning objectives

By the end of this chapter, you will be able to:

- Build an inverted index over a folder of cleaned DDR text.
- Answer keyword queries like "which reports mention losses?" in milliseconds instead of minutes.
- Explain why exact keyword matching, while fast and simple, will eventually miss things a drilling engineer would consider an obvious hit.

3.2. Operational Problem

You now have clean, expanded text for every DDR. The next question is the one an engineer actually asks out loud: *"Which reports mention losses?"* or *"Show me the report where they had to fish for something."* Opening files one by one doesn't scale past a handful of reports — you need an index.

3.3. Example DDR extract

> ℹ Real query and expected hit
>
> ```
> Query: "losses"
> Expected hit: FORGE-16A-78-32_Drilling_019_2020-11-07.txt
>   → "Mud fluids seepage losses in six hours 9 bbls"
> ```

That's report #19 — a minor, real seepage-loss event, easy to miss by eye across 76 reports, trivial to find once indexed.

3.4. Theory

An **inverted index** flips the natural document → words relationship around: instead of asking "what words are in this document," it stores, for every word, *which documents contain it.* Looking up "losses" becomes a single dictionary lookup instead of scanning every file. This is the same core idea behind every search engine, from `grep` to Google — the difference is scale and ranking sophistication, not the fundamental mechanism.

> Engineering Translation: Index
>
> An **index** here is exactly what it sounds like from the front of a manual: a table of contents for reports. Instead of "chapter 4, see page 112," it says "the word 'losses', see report_019." Looking a word up in that table is instant; reading every report to find the same word is not.

Tokenization matters here: "losses" and "LOSSES" should be the same token, and punctuation shouldn't create spurious tokens (`6,507'` should not prevent `6507` or nearby words from matching cleanly).

```
documents                              inverted index
------------------------------         --------------------------
report_038: "...became stuck..."       "stuck"   -> {report_038}
report_019: "...seepage losses..."     "pipe"    -> {report_038}
report_050: "...milled up lost..."     "losses"  -> {report_019}
                                        "milled"  -> {report_050}

query "stuck" -> look up "stuck" -> {report_038}   (one dict lookup,
                                                     not ten file scans)
```

3.5. Implementation

3.5.1. Step 1: split text into searchable words

3.6. What problem are we solving?

Before we can look anything up, every report's text needs to be broken into a consistent set of words — so "Losses", "losses," and "LOSSES" all count as the same match.

3.7. Inputs

- Any string of report text.

3.8. Expected Output

A list of lowercase words with punctuation stripped out, for example:

```
["mud", "fluids", "seepage", "losses", "in", "six", "hours", "9", "bbls"]
```

```
# code/chapter_03/keyword_search.py
import re

def tokenize(text: str) -> list[str]:
    return re.findall(r"[a-z0-9]+", text.lower())
```

3.9. What just happened?

This lowercases the whole report, then pulls out every run of letters or digits as one word, throwing away punctuation, quotes, and apostrophes. `6,507'` becomes just `6507` — no comma, no foot mark — so it can be matched consistently whichever way it was typed in the original report.

3.9.1. Step 2: build the lookup table

3.10. What problem are we solving?

Build a lookup table that remembers which reports contain each word, so a search never has to open and re-scan every file.

3.11. Inputs

- A folder of cleaned, expanded report text: `datasets/ddr_text_expanded/`.

3.12. Expected Output

A dictionary mapping each word to the set of report filenames that contain it, for example:

```
{"losses": {"FORGE-...-32_Drilling_019_2020-11-07.txt"},
 "stuck":  {"FORGE-...-32_Drilling_038_2020-11-26.txt"},
 ...}
```

```python
from collections import defaultdict
from pathlib import Path

def build_index(text_dir: Path) -> dict[str, set[str]]:
    index: dict[str, set[str]] = defaultdict(set)
    for path in sorted(text_dir.glob("*.txt")):
        tokens = set(tokenize(path.read_text(encoding="utf-8")))
        for token in tokens:
            index[token].add(path.name)
    return index
```

3.13. What just happened?

We built a lookup table that remembers which reports contain each word. For every report, we found its unique words and, for each one, noted "this word shows up in this report." The result is one table you can check instead of seventy-six files you'd otherwise have to open.

3.13.1. Step 3: answer a query

3.14. What problem are we solving?

Given a query like "stuck pipe," find every report that contains all of those words — instantly, using the lookup table instead of opening a single file.

3.15. Inputs

- The lookup table from Step 2.
- A query string, e.g. `"stuck pipe"`.

3.16. Expected Output

The set of filenames containing every word in the query, for example `{"FORGE-...-32_Drilling_038_2020-11-26.txt"}` for the query `"stuck"`.

```python
def search(index: dict[str, set[str]], query: str) -> set[str]:
    query_tokens = tokenize(query)
    if not query_tokens:
        return set()
    results = index.get(query_tokens[0], set()).copy()
    for token in query_tokens[1:]:
        results &= index.get(token, set())
    return results
```

3.17. What just happened?

For each word in the query, we looked up its list of reports in the table, then kept only the reports that appeared on *every* one of those lists. That's why this is called an AND query: a report only matches if it contains every word you searched for, not just one of them. The challenge exercise below asks you to add an OR option too.

3.18. Production Reality

This index lives entirely in memory and rebuilds itself every time you run the script — fine for 76 reports, and still fine for a few thousand. A real multi-well archive stretches this in ways worth knowing about early:

- thousands of reports mean rebuilding the index on every run gets slow — real systems save the index to disk or a database instead
- OCR'd scanned reports (Chapter 6) introduce garbled tokens that were never real words, bloating the index with noise
- singular/plural and tense variants (`loss` vs `losses`, `drilled` vs `drilling`) are treated as completely different words by this simple tokenizer, so a search can still miss obvious matches
- numbers and units (`6,507'` vs `6507 ft`) can drift apart across reports from different software or operators

None of this changes the core idea — a lookup table beats scanning every file — but it's why production search systems add stemming, on-disk indexes, and cleanup steps this chapter deliberately leaves out.

3.19. Practical exercise

Try it yourself: Build the index over `datasets/ddr_text_expanded/` and run `search(index, "fishing")`.

You'll know it worked when: the result set contains **two** reports, not one: `FORGE-16A-78-32_Drilling_049_2020-12-07.txt` (the day the crew picked up a fishing BHA and logged `ACTIVITY PLANNED: FISHING OPERATIONS`) and `FORGE-16A-78-32_Drilling_050_2020-12-08.txt` (the day they actually milled up lost pieces of bit). If you expected only one result, that's a useful surprise: the word "fishing" genuinely appears in both the planning day and the execution day, and an AND/OR keyword index has no concept of "the day it actually happened" versus "the day it was planned" — it just finds every document containing the word.

3.20. Field notes

⚠ Field notes: the search that misses the day after

Query: `"stuck pipe"`
Result: one hit — report #38 (2020-11-26), the day the assembly actually got stuck. Report #39, the very next day, is invisible to this query.
Why: report #39's own words are:

- "Work tight hole at 6,526'."
- "Due to high torque decision to pull out of hole"
- "Hole drag from 6,050' to 5,901' no issues"

but the word "stuck" never appears anywhere in report #39. `pipe` does (buried in a BHA table row, `Drill Pipe-5'' HWDP`), but the AND search still fails because `stuck` alone has zero matches outside report #38.
Lesson: an engineer researching *"how did the stuck-pipe situation on this well develop"* would want report #39 too — tight hole, high torque, and a decision to pull out of hole are exactly the kind of continued-risk language that matters the day after an incident. Keyword search can't surface it, because report #39 never uses the word the query asked for. This isn't a bug in the code above — it's the fundamental limit of matching on exact words at all. Chapter 4 comes back to this same query.

3.21. Challenge exercise

Challenge: Add phrase search (query tokens must appear *adjacent* and in order, not just co-occur anywhere in the document) and an OR operator. Test `search(index, "stuck")` against report #38, then try `"packers OR fishing"` and confirm it returns both report #49 (packers) and #50 (fishing). A reference solution is in `code/chapter_03/challenge/`.

3.22. Key takeaways

- An inverted index turns "search every file" into "look up a word," which is the difference between milliseconds and minutes at scale.
- Tokenization choices (case folding, punctuation handling) determine what counts as a match — get this wrong and searches silently fail.
- Exact keyword matching is fast, simple, and fundamentally limited: it cannot find report #49 ("packers did not set… pick up Fishing BHA") when you search "lost circulation," even though report #19's "seepage losses" line and report #49's packer failure are both, in a loose sense, "things going wrong downhole" — a human would connect them faster than exact keywords can.

3.23. Repository files

File	Purpose
`code/chapter_03/keyword_search.py`	Inverted index and AND-query search
`notebooks/chapter_03_explore.ipynb`	Interactive companion notebook

3.24. Why keyword search fails

Question: *"Show me previous stuck-pipe trouble on this well."*

Keyword search for "stuck pipe" finds report #38 — the day it actually happened — and stops there. It misses report #39, filed the very next day, because that report never uses the word "stuck." Instead it says:

- "Work tight hole"
- "high torque decision to pull out of hole"
- "Hole drag from 6,050' to 5,901"'

That's the same operational story — continued stuck-pipe risk, one day later — written in entirely different words. Keyword search cannot connect them, because it only ever checks for exact spelling. It doesn't know what any word *means*, only whether it's present or absent.

That gap — matching meaning, not just spelling — is exactly what Chapter 4 solves, using a technique called embeddings.

3.25. What can you do now that you couldn't do before?

You can search an archive of reports for exact words or phrases and get an answer in milliseconds, instead of opening files one at a time.

3.26. Suggested next step

Keyword search is exact and unforgiving — it doesn't know that "high torque decision to pull out of hole" and "stuck pipe" describe related trouble. Chapter 4 replaces exact matching with semantic search, so retrieval works by meaning, not just spelling — and comes back to this exact "stuck pipe" query to see how much that actually fixes.

4. Semantic Search

```
Searchable Archive
 ↓
Search by Meaning
```

4.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain, without heavy math, what a text embedding is and why nearby vectors mean similar meaning.
- Generate embeddings for a folder of DDR text using `sentence-transformers`.
- Retrieve the most semantically similar chunks to a query using cosine similarity, without any exact keyword overlap required.
- Recognise that semantic search over whole documents is an improvement, not a complete fix — and that granularity is what actually determines how well it works.

4.2. Operational Problem

Chapter 3 ended on a real miss: the query `"stuck pipe"` found report #38 (the actual incident) but was blind to report #39, which describes tight hole, high torque, and a decision to pull out of hole — real continued-risk language, written the very next day — without ever using the word "stuck." An engineer reading both reports would say "yes, obviously, day two of the same problem." A keyword search can't see that at all. Let's find out, honestly, how much of that gap semantic search actually closes.

4.3. Example DDR extract

> **i** The same query, now by meaning

```
Query: "stuck pipe"  (same query Chapter 3 used)

Keyword search (Chapter 3): report #39 not in results at all.

Semantic search, whole-report embeddings (this chapter): report #39
  ranks 5th out of 10 - findable, but not prominent. Report #38 (the
```

> actual incident) ranks 2nd.

4.4. Theory

An **embedding model** converts a piece of text into a fixed-length vector of numbers — typically a few hundred dimensions — such that texts with similar meaning end up as vectors that are close together in that space. You don't need to understand the neural network that produces this vector; you need to understand what it gives you: a numeric representation of meaning you can compare with simple arithmetic.

> **Engineering Translation: Vector / embedding**
>
> Think of an embedding as GPS coordinates for meaning. Instead of latitude and longitude, it's a few hundred numbers — but the idea is the same: two sentences with similar meaning end up with "coordinates" close together, the same way two rigs in the same field end up with similar latitude and longitude, even if their names have nothing in common.

Cosine similarity measures the angle between two vectors — 1.0 for identical direction, 0 for unrelated, negative for opposite. It's the standard way to compare embeddings because it ignores vector *length* and focuses purely on *direction*, which is what carries the meaning here.

> **Engineering Translation: Cosine similarity**
>
> Cosine similarity is a compass bearing, not a ruler. It asks "which way is this thing pointing?", not "how big is it?" — so a short sentence and a long paragraph about the same topic can still score as pointing the same direction, the same way two surveys can report the same azimuth regardless of how long each run was.

We use `sentence-transformers` (Reimers and Gurevych 2019) with a small, fast model (`all-MiniLM-L6-v2`) — good enough to prove the concept and small enough to run on a laptop CPU. Chapter 8 replaces the brute-force search here with a proper vector database once the corpus grows past what fits comfortably in memory.

```
text ("stuck pipe")
   ↓
embedding model (all-MiniLM-L6-v2)
   ↓
vector: [0.02, -0.15, 0.33, ...]   (384 numbers)
   ↓
compare against every document's vector via cosine similarity
   ↓
nearby vectors = similar meaning, regardless of shared vocabulary
```

4.5. Implementation

4.5.1. Step 1: load every report's text

4.6. What problem are we solving?

Get every report's text into memory, alongside its filename, so we know which score belongs to which report later.

4.7. Inputs

- A folder of cleaned report text, e.g. `datasets/ddr_text_expanded/`.

4.8. Expected Output

Two matching lists: report filenames, and each report's full text.

```
# code/chapter_04/semantic_search.py
from pathlib import Path

def load_chunks(text_dir: Path) -> tuple[list[str], list[str]]:
    filenames, texts = [], []
    for path in sorted(text_dir.glob("*.txt")):
        filenames.append(path.name)
        texts.append(path.read_text(encoding="utf-8"))
    return filenames, texts
```

4.9. What just happened?

Nothing about meaning yet — this just reads every text file into memory and keeps a matching list of filenames, so that whatever score we calculate next can be traced back to the report it came from.

4.9.1. Step 2: convert text into vectors

4.10. What problem are we solving?

Turn each report's text into a numeric representation of its meaning, so that "similar meaning" can be measured with arithmetic instead of exact word matching.

4.11. Inputs

- `MODEL_NAME = "all-MiniLM-L6-v2"`, a pre-trained embedding model.
- The list of report texts from Step 1.

4.12. Expected Output

One 384-number vector per report, all packed into a single array.

```
import numpy as np
from sentence_transformers import SentenceTransformer

MODEL_NAME = "all-MiniLM-L6-v2"

def embed_texts(model: SentenceTransformer, texts: list[str]) -> np.ndarray:
    embeddings = model.encode(texts, normalize_embeddings=True)
    return np.asarray(embeddings)
```

4.13. What just happened?

The model read each report and produced its "GPS coordinates for meaning" — one vector per report. `normalize_embeddings=True` scales every vector to the same length, so later we can compare direction only, which is exactly what cosine similarity needs.

4.13.1. Step 3: rank reports against a query

4.14. What problem are we solving?

Given a query like "stuck pipe," find which reports are closest in meaning — not which reports contain those exact words.

4.15. Inputs

- The query string.
- The array of report vectors from Step 2.

4.16. Expected Output

A ranked list of `(filename, score)` pairs, highest similarity first.

```
def search(model: SentenceTransformer, query: str, filenames: list[str],
           embeddings: np.ndarray, top_k: int = 3) -> list[tuple[str, float]]:
    query_vec = model.encode([query], normalize_embeddings=True)[0]
    scores = embeddings @ query_vec  # cosine similarity, since vectors are
↪   normalized
    top_indices = np.argsort(-scores)[:top_k]
    return [(filenames[i], float(scores[i])) for i in top_indices]
```

4.17. What just happened?

The query got turned into its own vector, then compared against every report's vector at once. Because every vector was normalized to the same length in Step 2, one matrix multiplication (`embeddings @ query_vec`) gives back the cosine similarity for every report simultaneously — that's the "arithmetic" that replaces exact word matching.

Run this against all ten sample reports with `query="stuck pipe"` and `top_k=10` (the whole ranking, not just the top few), and you get the real numbers behind the callout above:

```
1. 0.2416  Completion_003_2021-01-06
2. 0.1978  Drilling_038_2020-11-26   <- the actual stuck-pipe day
3. 0.1713  Drilling_049_2020-12-07
4. 0.1657  Drilling_003_2020-10-22
5. 0.1498  Drilling_039_2020-11-27   <- tight hole / high torque, findable now
6. 0.1406  Drilling_048_2020-12-06
7. 0.1342  Drilling_019_2020-11-07
8. 0.1326  Drilling_037_2020-11-25
9. 0.1319  Drilling_050_2020-12-08
10. 0.1169 Drilling_036_2020-11-24
```

Report #39 went from *absent* (Chapter 3) to *rank 5 of 10* — a real improvement, worth having. But it's not a clean win: the scores are bunched close together (0.12–0.24), and a busy engineer scanning only the top 2 or 3 results would still miss it.

4.18. Production Reality

This chapter runs the embedding model locally, on a laptop CPU, against ten short reports. Real deployments hit constraints this setup never sees:

- some teams call a hosted embedding API instead of running a model locally — which means sending report text, possibly confidential well data, to a third party. That's a data-governance conversation, not just a technical choice, and it's worth having before it's a surprise.

- embeddings from two different model versions aren't comparable — if you re-embed your archive with a newer model, old and new vectors can't be compared against each other, and everything needs re-embedding together.
- embedding a full multi-well archive (thousands of reports, not ten) takes real time and, for hosted APIs, real money — a cost that scales with archive size, not query volume.
- a similarity score like 0.24 is only meaningful *relative to other scores in the same search* — it is not "24% similar" in any absolute sense, and comparing scores across different queries or models is not valid.

4.19. Field notes

⚠ Field notes: why whole-document embeddings are noisy

Query: `"stuck pipe"`, same as above.
Result: report #39 ranks 5th of 10 when you embed each *entire report* as one vector — an improvement over keyword search's zero, but not a clean fix.
Why: each report is one page, but that page holds seven distinct data tables (casing, mud, drill bits, pumps, BHA, survey data, consumables) plus the narrative time breakdown. Embedding the whole thing blends a handful of sentences of relevant narrative with a few hundred words of numeric table content. The relevant signal — "Work tight hole… high torque… pull out of hole" — is a small fraction of what actually goes into the vector.
Isolate just the narrative lines instead of the whole report, and the picture changes completely:

```
0.4868  report #38 - "Work pipe, circulate lube sweep, work tool
                      back in position, Pipe free"
0.3359  report #38 - "During the slide lost tool face and became
                      assembly became stuck"
0.2402  report #39 - "Due to high torque decision to pull out of hole"
0.2329  report #49 - "Attempted multiple times to set packers..."
0.2143  report #39 - "Work tight hole at 6,526 feet."
0.1892  report #39 - "Hole drag from 6,050' to 5,901' no issues"
0.1892  report #36 - "Trip out of hole with BHA #17 core assembly..."
        (genuinely unrelated coring operation, same score as above)
```

At line granularity, report #39's high-torque line clearly separates from genuinely unrelated content (0.24 vs. 0.19) — something whole-document embedding couldn't show at all.
Lesson: semantic search doesn't fail because the *technique* is wrong — it fails here because the *unit of retrieval* is too coarse. This is exactly the problem Chapter 7's chunking solves, and it's worth remembering the next time a semantic search "isn't working": check what's actually inside each embedded vector before blaming the model.

4.20. Practical exercise

Try it yourself: Embed all ten sample DDRs and run `search(model, "stuck pipe", filenames, embeddings, top_k=10)`.

You'll know it worked when: `FORGE-16A-78-32_Drilling_039_2020-11-27.txt` appears somewhere in your ranked results — unlike Chapter 3, where it didn't appear at all — even if it isn't near the top.

4.21. Challenge exercise

Challenge: Reproduce the Field Notes line-level result yourself. Manually pull three lines of text — report #39's `"Due to high torque decision to pull out of hole"`, report #39's `"Hole drag from 6,050' to 5,901' no issues"`, and report #36's `"Trip out of hole with BHA #17 core assembly, lay down core barrels"` (a genuinely unrelated line) — embed just those three strings, and confirm the high-torque line scores meaningfully higher against `"stuck pipe"` than the other two, which should score close to each other. A reference solution is in `code/chapter_04/challenge/`.

4.22. Key takeaways

- Embeddings represent meaning as geometry: similar meaning, nearby vectors.
- Cosine similarity is the standard comparison because it isolates direction (meaning) from magnitude.
- Semantic search over whole documents is a real improvement on keyword search — report #39 goes from unfindable to rank 5 of 10 — but "an improvement" and "solved" are different claims. Don't oversell it.
- Granularity is often the actual lever, not the embedding model. The same query, same model, same text — just isolated to individual narrative lines instead of a whole noisy report — turns a middling rank-5 result into a clear, checkable win.
- Brute-force cosine search (a matrix multiply) is entirely adequate at ten documents. It stops being adequate well before you reach the full 76-report archive's scale — that's Chapter 8's problem to solve.

4.23. Repository files

File	Purpose
`code/chapter_04/semantic_search.py`	Embedding generation and brute-force cosine search
`notebooks/chapter_04_explore.ipynb`	Interactive companion notebook

4.24. What can you do now that you couldn't do before?

You can retrieve relevant report passages by meaning instead of exact wording — finding report #39's continued-risk language even though it never says "stuck" — and you know precisely why the result isn't perfect yet: granularity, not the model, is the real bottleneck.

4.25. Suggested next step

You can now retrieve relevant passages by meaning, and you've learned the honest limits of doing it at whole-document granularity. Chapter 5 turns retrieval into an answer: given a question, retrieve the right evidence, and generate a response that cites exactly where each claim came from. Chapter 7 comes back to fix the granularity problem directly.

5. First RAG System

```
Question
   ↓
Retriever
   ↓
LLM
   ↓
Answer + Citations
```

5.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain retrieval-augmented generation (RAG) as two separate, inspectable steps: retrieve, then generate.
- Assemble retrieved chunks into a prompt that forces a language model to answer only from the evidence it's given.
- Produce an answer with an explicit, per-claim evidence list — report number and page — rather than an unsupported paragraph.

5.2. Operational Problem

Retrieval (Chapter 4) gets you the right passages. What an engineer actually wants is an answer: "*What led to the fishing operation on report #50?*" — not two ranked text chunks they have to read and synthesise themselves. But a generated answer that isn't traceable back to specific reports is worse than no answer at all in an engineering context, because it invites misplaced trust. This chapter builds the smallest system that does both: synthesise an answer, and show exactly where every part of it came from.

5.3. Example DDR extract

Target interaction, grounded in two real, adjacent reports

```
Question: What led to the fishing operation on report #50?

Answer: On report #49 (2020-12-07), the crew attempted multiple times
```

```
to set packers on BHA #32, but pressure readings showed the ball did
not seat and the packers failed to set. They tripped out with the
packer assembly and picked up a fishing BHA (#33) that same day. On
report #50 (2020-12-08), that fishing run milled up lost pieces of bit.

Evidence:
  FORGE-16A-78-32_Drilling_049_2020-12-07.pdf
  FORGE-16A-78-32_Drilling_050_2020-12-08.pdf
```

Both claims here are directly quoted from real report text — nothing in this answer was inferred beyond what the two reports state. This is the *target*. Keep reading — the practical exercise below checks whether the simple system this chapter builds actually reaches it.

5.4. Theory

RAG (retrieval-augmented generation) (Lewis et al. 2020) is exactly the two chapters you just built, chained together: retrieve relevant evidence (Chapter 4), then hand that evidence to a language model and ask it to answer *using only that evidence*. The model is not answering from what it memorised during training — it's answering from the specific passages you retrieved, which is what makes citation possible at all.

> Engineering Translation: LLM
>
> An **LLM** (large language model) is like a very well-read new hire: it has absorbed an enormous amount of general drilling and engineering writing, so it writes fluently and knows the vocabulary — but it has never seen *your* wells or *your* reports. Left to its own memory, it will guess. RAG is how you make sure it answers from the actual report in front of it, not from a guess dressed up in confident language.

> Engineering Translation: Prompt
>
> A **prompt** is the instructions-plus-evidence packet you hand that new hire before asking them to write anything — the equivalent of handing them the specific job's DDRs and a clear brief, instead of just asking "what usually happens in a fishing job?" and hoping they don't improvise.

The critical design decision is the prompt: instruct the model explicitly to answer only from the provided context, and to cite which source each part of the answer came from. This doesn't make hallucination impossible — Chapter 10 covers stronger mitigations — but it is the foundation everything else builds on.

Here's the complete pipeline you've actually built, chapter by chapter — every arrow below is real, working code you've already written:

```
DDR PDF
   ↓
```

```
Extraction               (Chapter 1 - pdfplumber)
   ↓
Cleaning                 (Chapter 2 - abbreviation expansion)
   ↓
Embeddings               (Chapter 4 - sentence-transformers)
   ↓
Retriever                (Chapter 4 - cosine similarity search)
   ↓
LLM                      (this chapter - prompt + generation)
   ↓
Answer + Citations
```

This is deliberately the simplest version of this pipeline that could possibly work — brute-force search, whole-document embeddings, no reranking, no traceability guardrails beyond a citation list. Part II rebuilds nearly every arrow in this diagram to survive a real, full-scale archive.

5.5. Implementation

5.5.1. Step 1: write the instructions the model must follow

5.6. What problem are we solving?

Make sure the model answers from the evidence you hand it, not from whatever it remembers from training — and make sure it tells you which report each part of the answer came from.

5.7. Inputs

- None yet — this is a fixed template, filled in for each question later.

5.8. Expected Output

A reusable prompt template with two blanks: the evidence, and the question.

```python
# code/chapter_05/first_rag.py
PROMPT_TEMPLATE = """Answer the question using ONLY the evidence below.
If the evidence doesn't contain the answer, say so - do not guess.
Cite which report each part of your answer comes from.

Evidence:
{evidence}

Question: {question}
Answer:"""
```

5.9. What just happened?

This is the "brief" you hand the model every single time: answer only from what's provided, admit it when the evidence doesn't cover the question, and always say which report backs each claim. Everything else in this chapter exists to fill in `{evidence}` and `{question}` correctly.

5.9.1. Step 2: assemble the evidence into that template

5.10. What problem are we solving?

Take the reports Chapter 4's retriever found and package them, labelled by filename, into the prompt template's `{evidence}` blank.

5.11. Inputs

- A question string.
- The retrieved `(filename, score)` pairs from Chapter 4's `search`.
- The full filename and text lists from Chapter 4's `load_chunks`.

5.12. Expected Output

One complete prompt string, evidence labelled by source filename, ready to send to a model.

```
from pathlib import Path

from semantic_search import embed_texts, load_chunks, search  # from Chapter 4

def build_prompt(question: str, retrieved: list[tuple[str, float]],
                 filenames: list[str], texts: list[str]) -> str:
    evidence_blocks = []
    for filename, _score in retrieved:
        text = texts[filenames.index(filename)]
        evidence_blocks.append(f"[{filename}]\n{text}")
    return PROMPT_TEMPLATE.format(
        evidence="\n\n".join(evidence_blocks),
        question=question,
    )
```

5.13. What just happened?

For every report Chapter 4's retriever returned, this tags its full text with its own filename in square brackets, then stitches all of them together into the `{evidence}` section of the prompt. Every fact the model can possibly cite is now labelled with exactly which report it came from.

5.13.1. Step 3: retrieve, then generate, then report the sources

5.14. What problem are we solving?

Chain retrieval and generation into one function: given a question, find the evidence, ask the model to answer from it, and hand back both the answer and the exact list of reports it was allowed to use.

5.15. Inputs

- A question string.
- The embedding model, filenames, texts, and embeddings from Chapter 4.
- `llm_call`: any function that takes a prompt string and returns the model's answer.

5.16. Expected Output

A tuple: the generated answer text, and the list of report filenames that were actually retrieved and shown to the model.

```
def answer_question(question: str, model, filenames, texts, embeddings,
                    llm_call) -> tuple[str, list[str]]:
    retrieved = search(model, question, filenames, embeddings, top_k=3)
    prompt = build_prompt(question, retrieved, filenames, texts)
    answer_text = llm_call(prompt)  # plug in your LLM of choice
    evidence = [filename for filename, _score in retrieved]
    return answer_text, evidence
```

5.17. What just happened?

This is the whole system end to end: retrieve the top matching reports, build a prompt from them, generate an answer, and return that answer alongside the exact evidence list it was built from — so you (or an engineer reviewing the answer) can always check the citation against the real reports, rather than taking the model's word for it.

`llm_call` is intentionally a plain function argument, not a hardcoded API client: plug in a local model, an API-based model, or — while you're first testing the retrieval and prompt logic — a stub that just echoes the evidence back, so you can verify the pipeline end to end before spending a single API call.

5.18. Production Reality

This chapter's prompt says "answer only from the evidence" — but an instruction in a prompt is a strong nudge, not a guarantee. Real systems have to plan around what happens when that nudge isn't enough:

- models don't follow instructions perfectly every time — occasionally one will still answer from general knowledge instead of the evidence, or blend the two without saying so
- the same question, asked twice, can produce two differently worded answers — LLM output isn't fully deterministic, which matters when someone asks "why did it say something different yesterday?"
- long evidence sections eventually hit the model's context window limit — a real archive's retrieved chunks can't just grow without bound
- every call to a hosted LLM API costs money and takes time; a system answering hundreds of engineering questions a day needs a cost and latency budget, not just a working prompt

None of this means RAG doesn't work — it means "answer only from evidence" is a design goal you build toward, not a switch you flip. Chapter 10 covers the guardrails that close this gap further.

5.19. Practical exercise

Try it yourself: Wire up `answer_question` with a stub `llm_call` that simply returns the evidence text, run it for "What led to the fishing operation on report #50?" with `top_k=3`, and print the evidence list.

You'll know it worked when: you can see exactly which three reports came back — and, before reading further, decide for yourself whether that list actually contains what the target answer above needs.

5.20. Field notes

Field notes: the report the question is about isn't in the evidence

Query: `"What led to the fishing operation on report #50?"`, `top_k=3`.
Result: retrieval returns report #49 (rank 1) plus two other reports — but **not report #50**, the report the question is literally about. The full ranking:

```
1. 0.4409  Drilling_049_2020-12-07   <- packers fail (correctly retrieved)
2. 0.3523  Drilling_038_2020-11-26
3. 0.3270  Drilling_048_2020-12-06
...
7. 0.3106  Drilling_050_2020-12-08   <- the fishing report itself
```

Why: report #50's own text — bit-mill fishing narrative buried among casing tables, mud tables, and a full BHA component list — is, at whole-document granularity, more similar to *several other reports'* tables than it is to a question about fishing. This is the same granularity problem Chapter 4 already flagged; here it has a sharper consequence.
Lesson: this is exactly the failure mode this chapter's own Key Takeaways warn about — and it's a dangerous one specifically *because* the question names "report #50" directly. A careless `llm_call` implementation could echo that report number back in its answer without ever having actually retrieved report #50's content, producing a citation-shaped sentence that *looks* grounded and isn't. Always check what's actually in the evidence list — not what the question happens to mention — before trusting an answer. Chapter 7's chunking and Chapter 9's hybrid retrieval are what actually close this gap; Part I's simple system doesn't, and pretending otherwise would defeat the entire point of this book.

5.21. Challenge exercise

Challenge: Connect a real LLM (local via `transformers`/`ollama`, or a hosted API) as `llm_call`, and add a check that rejects the answer if it mentions a report filename that wasn't in the retrieved evidence list — a first, crude hallucination guard. Then ask it a question the ten-report sample archive genuinely can't answer (e.g. "what happened on report #100?") and confirm it says so rather than inventing an answer. A reference solution is in `code/chapter_05/challenge/`.

5.22. Key takeaways

- RAG is retrieval, then generation — two separate, individually debuggable steps, not one opaque black box.
- If retrieval is wrong, generation cannot save you — always verify the evidence list before trusting the generated answer.
- A prompt that instructs "answer only from evidence, cite sources" is necessary but not sufficient for trustworthy answers. Part II builds the guardrails that make it sufficient.

5.23. Repository files

File	Purpose
`code/chapter_05/first_rag.py`	Prompt assembly and evidence-tracked question answering
`notebooks/chapter_05_explore.ipynb`	Interactive companion notebook

5.24. What can you do now that you couldn't do before?

You can ask an engineering question in plain English and get back a generated answer with an explicit, checkable list of exactly which reports it came from — not just ranked passages you'd still have to read and synthesise yourself.

5.25. Suggested next step

What you've built in Part I works — on ten clean, digital, hand-picked reports out of the full 76-report Utah FORGE archive. Part II industrialises this system against the reality of a full archive: retrieval that has to be both fast and precise across every report, and answers that have to survive scrutiny from someone who wasn't in the room when they were generated. Chapter 6 starts with the first thing that breaks in a typical real-world archive: scanned reports with no extractable text at all.

Part II.

Part II — Industrialising the System

6. Scanned Reports and OCR Quality Gates

```
Scanned PDF
 ↓
Trusted Text
```

6.1. Learning objectives

By the end of this chapter, you will be able to:

- Recognise when a DDR PDF has no extractable text and needs OCR.
- Run OCR on a scanned page and understand why the raw output is often too noisy to index as-is.
- Build a quality gate that rejects bad OCR output *before* it enters your search index, rather than trying to fix it after the fact.

6.2. Operational Problem

Can you trust the text this archive extracted — even from a report that was scanned instead of born digital? Every one of the 76 Utah FORGE reports used in this book is digital-native — the archive's QA/QC pass confirms 0 unparsed PDFs across 227 source files. You got lucky: WellEz, the software that generated these reports, embeds real character data on every page. Not every archive is this well-behaved. Faxed reports, reports scanned from paper files, and photos of a logbook page all produce PDFs with no character data at all — just a picture of text. Chapter 1's `extract_text()` handled this gracefully (an empty string, not a crash), but a real, larger archive needs more than graceful failure: it needs OCR to recover the text, and a way to know whether to trust what OCR gives back.

6.3. Example DDR extract

OCR output before and after quality gating

```
Clean digital extraction (this is what every Utah FORGE report gives):
  "During the slide lost tool face and became assembly became stuck"

Degraded OCR output from a hypothetical poor scan of the same line:
  "Dur1ng the s1ide los+ t00l face and became assembIy became stu(k"
```

> The second block is exactly the kind of text a naive pipeline would embed and index without complaint — and exactly the kind of text a quality gate should catch and route to a review queue instead.

6.4. Theory

Getting OCR to run is the easy part — `pytesseract`, cloud OCR APIs, and PDF-rendering libraries all do a competent job of turning a page image into text. The hard part, and the one that actually determines whether your downstream system is trustworthy, is deciding **whether to trust the output at all**. A quality gate scores OCR text against a handful of cheap heuristics — is there enough text, is the alphabetic-to-symbol ratio sane, is there suspicious repeated garbage — and rejects anything that fails, routing it to manual review instead of silently indexing noise.

> Engineering Translation: Quality gate
>
> A **quality gate** is a mud check before you commit to pumping downhole: a handful of cheap tests run before you trust what comes next. Passing doesn't prove the text is perfect, the same way a good mud check doesn't prove there's no problem downhole — it just screens out the obviously bad cases before they cost you.

> Engineering Translation: Flag
>
> A **flag** here is one checkbox on an inspection form — "too little text" either applies or it doesn't. One flag firing on its own usually isn't grounds for rejection (a short, sparse-but-real report can trip one check honestly); it's when *several* flags fire together that the page is actually suspect.

This chapter's companion pipeline, **DDR_UTAH_FORGE**, implements exactly this pattern in `src/rag_pdf/ocr_quality.py` — even though, as noted above, this particular archive never needs to exercise it. That's by design: the quality gate runs unconditionally on every extracted page, digital or scanned, as a defensive check. Its `evaluate_ocr_quality()` function checks four independent flags — `low_text_density`, `high_symbol_ratio`, `repeated_garbage`, `low_line_count` — and rejects a page only once **two or more** flags fire, so no single noisy-but-legitimate page (e.g. a short report with a lot of numeric data) gets discarded on one false positive.

```
extracted text
   ↓
compute four flags
(low_text_density, high_symbol_ratio,
 repeated_garbage, low_line_count)
   ↓
count how many are True
   ↓
2 or more active?
   ↓
```

```
yes -> reject, flag for review
no  -> accept into index
```

6.5. Implementation

Build a simplified version of the same idea.

6.6. What problem are we solving?

Decide whether a page's extracted text is trustworthy enough to add to the search index, or whether it should be set aside for a human to check instead.

6.7. Inputs

- Extracted text (from OCR, or from Chapter 1's digital extraction).
- Thresholds: minimum characters, minimum alphabetic words, maximum symbol ratio, and how many flags must fire before rejecting.

6.8. Expected Output

A dictionary reporting the decision and which specific checks failed, for example:

```
{"reject_ocr": True, "flags": {"low_text_density": True, "high_symbol_ratio": True}}
```

```python
# code/chapter_06/ocr_quality_gate.py
import re

def evaluate_ocr_quality(text: str, min_chars: int = 200,
                          min_alpha_words: int = 30,
                          max_symbol_ratio: float = 0.35,
                          reject_min_flags: int = 2) -> dict:
    alpha_tokens = re.findall(r"[A-Za-z]+", text)
    non_ws = [c for c in text if not c.isspace()]
    noisy = sum(1 for c in non_ws if not c.isalnum() and c not in
↪   set("%.,()/-:'"))
    symbol_ratio = noisy / max(len(non_ws), 1)

    flags = {
        "low_text_density": len(text) < min_chars or len(alpha_tokens) <
         ↪ min_alpha_words,
        "high_symbol_ratio": symbol_ratio > max_symbol_ratio,
    }
```

```
    active = [k for k, v in flags.items() if v]
    return {"reject_ocr": len(active) >= reject_min_flags, "flags": flags}
```

6.9. What just happened?

This measures two things about the text: how much of it is real, readable words (`low_text_density`), and how much of it is odd symbols that shouldn't be there (`high_symbol_ratio`). Each check is a simple yes/no. Only when two or more of those checks say "yes, this looks wrong" does the function recommend rejecting the page — a single unusual character never sinks an otherwise legitimate report.

Test it against a real Utah FORGE report's clean text (it should pass easily — that text has a healthy word count and almost no noise symbols) and against the corrupted example above (it should fail on both flags).

The full production version — including the repeated-garbage detector and line-count check — is in the companion repository at `src/rag_pdf/ocr_quality.py`, alongside `src/rag_pdf/rotation_handler.py` for detecting and correcting upside-down or sideways scans, and `src/rag_pdf/table_ocr_handoff.py` for the case where a *table* (not narrative text) needs OCR fallback.

6.10. Production Reality

This chapter's quality gate answers one narrow question: should this specific page be trusted? It doesn't answer what happens next. A real archive needs a plan for that too:

- a flagged page needs an actual person to review it — a gate only earns its keep if someone owns the review queue it creates, otherwise flagged pages just pile up unread
- rotated or upside-down scans (common when a paper report goes into a scanner sideways) need correcting *before* OCR runs at all — that's a separate step from quality scoring, handled by the companion `rotation_handler.py`
- handwritten margin notes on an otherwise-typed report often don't OCR into anything usable — the gate will correctly flag them as low-density, but "OCR failed" and "this needs a human to transcribe handwriting" call for different follow-up
- an archive spanning multiple regions or operators may include reports in more than one language, which changes which OCR language pack — and which quality thresholds — actually apply

None of this shows up in the all-digital Utah FORGE archive, which is exactly why it's easy to forget about until a real archive containing any of the above lands on your desk.

6.11. Practical exercise

Try it yourself: Run `evaluate_ocr_quality()` against the extracted text of report #38 (clean) and the degraded example string above.

You'll know it worked when: the real report text passes cleanly, and you can explain, from the flags returned, exactly why the degraded text fails.

6.12. Field notes

Field notes: checking the gate doesn't cry wolf

Action: run `evaluate_ocr_quality()` against all ten real sample reports, including report #3 — the sparsest one in the archive, written before the well was even spudded, with most of its tables entirely empty.
Result: every single report passes cleanly. Report #3, the thinnest of the ten, still has 2,235 characters and 361 alphabetic words — comfortably clear of the 200-character / 30-word thresholds:

```
chars=2235 alpha_words=361 symbol_ratio=0.006  reject=False
```

Why: even a nearly-empty DDR carries a full page of section headers, column labels, and boilerplate ("WELL/JOB INFORMATION", "PRESENT OPERATIONS", table column names) regardless of how little actually happened that day. That structural text alone is enough to clear the density thresholds.
Lesson: a quality gate is only trustworthy if it doesn't also reject legitimate sparse content — a gate that flagged every quiet day as "probably OCR garbage" would be useless in practice, burying real pages under false alarms. Checking the gate against your *most* sparse real documents, not just your noisiest fake ones, is how you find that out before it costs you.

6.13. Challenge exercise

Challenge: This archive is 100% digital, so you can't test the gate against a real scanned Utah FORGE page. Instead, write a small function that synthetically corrupts clean text (swap random characters for lookalikes, inject repeated tokens) at increasing severity, and find the corruption level at which `evaluate_ocr_quality()` starts rejecting it. This is a legitimate way to stress-test a quality gate before you ever encounter the real noisy data it's meant to catch.

6.14. Key takeaways

- OCR quality gating is cheap, heuristic, and worth running unconditionally — even an archive that's currently 100% digital may not stay that way, and the check costs almost nothing on clean text.

- Require multiple independent signals to agree before rejecting a page; a single heuristic alone produces too many false positives.
- Reject-and-flag-for-review beats silently-index-anyway every time traceability matters more than completeness.

6.15. Repository files

File	Purpose
`code/chapter_06/ocr_quality_gate.py`	Simplified OCR quality gate
`DDR_UTAH_FORGE/src/rag_pdf/ocr_quality.py`	Production quality gate (companion repo)
`DDR_UTAH_FORGE/src/rag_pdf/rotation_handler.py`	Scan rotation detection (companion repo)

6.16. What can you do now that you couldn't do before?

You can tell, automatically, whether a page of extracted or OCR'd text is trustworthy enough to search on — and route the pages that aren't to a human for review, instead of silently indexing garbage alongside good reports.

6.17. Suggested next step

Once you trust the text coming out of OCR and digital extraction alike, the next question is how to break it into pieces small enough to embed and retrieve — without cutting a stuck-pipe narrative in half across two chunks. That's Chapter 7.

7. Chunking That Respects Report Structure

```
Whole Report
 ↓
Retrievable Chunks
```

7.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why chunk size is measured in tokens, not characters or words.
- Implement token-based chunking with overlap using `tiktoken`.
- Implement segment-aware chunking that breaks at natural report boundaries (section headers) instead of at an arbitrary character count.

7.2. Operational Problem

Why does the exact sentence you need keep landing right on a chunk boundary? Chapters 1–5 embedded each report as a single unit — fine for a ten-report sample corpus, unworkable at the full archive's scale. A single Utah FORGE report already has seven distinct sections (WELL/JOB INFORMATION, BOP, CASING, MUD, DRILL BITS, PUMPS, BHA, SURVEY DATA, CONSUMABLES, TIME BREAKDOWN) packed onto one page. Embed too much of that in one vector and the embedding blurs the casing programme together with the stuck-pipe narrative, hurting retrieval precision. Split naively at a fixed character count and you'll cut report #38's stuck-pipe sentence in half mid-word, right across a chunk boundary — the exact passage you most need to retrieve becomes the two chunks least likely to score well.

7.3. Example DDR extract

> **ℹ Why naive splitting fails, using real report #38 text**
>
> ```
> Naive fixed-character split, mid-sentence:
>
> Chunk A: "...23:30 04:00 4.5 Drill From 6,360' to 6,507', (147') Total,
>           32.6' feet per hour. WOB 20 TO 35k, Rotary 50, Torque 6,500,
>           SPP 3200-3400 GPM 560, DIFF 200-300psi During the slide lost
>           tool face and became assembly became st"
> ```

```
Chunk B: "uck 04:00 06:00 2.0 Work pipe, circulate lube sweep, work
          tool back in position Pipe free"
```

A query about "what happened when the assembly got stuck" now has its key evidence split across two separately-scored chunks — neither of which alone contains the complete sentence.

7.4. Theory

Two ideas, used together, fix this:

1. **Token-based sizing.** Language models and embedding models operate on tokens, not characters — and token count is what actually determines whether a chunk fits a model's context window and how much a given chunk "costs" downstream. Chunking by token count (with a library like `tiktoken`) is more accurate than chunking by character or word count.
2. **Segment-aware boundaries.** Rather than cutting at a fixed token count regardless of content, detect natural boundaries — section headers like `TIME BREAKDOWN` or `SURVEY DATA`, which every Utah FORGE report uses consistently — and prefer to split there. A chunk that ends at a genuine section break is far more likely to be a self-contained, retrievable unit of meaning than one that ends mid-sentence because a counter hit its limit.

Engineering Translation: Token

A **token** is the unit a language model actually counts in — roughly a word or a word-fragment, not a character and not quite a word either. It's the same idea as a rig measuring progress in footage drilled, not characters typed on the report: footage is the unit that actually determines what the operation costs and how far along it is.

Engineering Translation: Overlap

Overlap between chunks is like re-surveying the last few feet of the previous run before starting the next one: a small deliberate repeat at the seam, so nothing that mattered right at the boundary gets lost between the two pieces.

The companion pipeline, **DDR_UTAH_FORGE**, implements both in `src/rag_pdf/chunking.py`. `chunk_text_by_tokens()` does token-based splitting with configurable overlap (so context isn't lost entirely at a boundary), falling back to a word-based approximation if `tiktoken` isn't available. `split_text_for_segment_aware_chunking()` goes further: it walks the text line by line, detects boundary patterns, and returns a list of `SegmentBlock` objects — each one bundling together a title, its text, and a `boundary_type` (`MATCH`, `INSERT`, or `CONTINUATION`) recording *why* the segment starts where it does, the same way an equipment spec sheet bundles a component's dimensions together instead of scattering them as loose numbers.

Run against the real 76-report archive, this pipeline's chunking produces **2,943 chunks** — an average of about 39 chunks per report, reflecting just how much structured content (header, seven data tables, and a narrative time breakdown) each single-page DDR actually contains.

Token-based chunking with overlap looks like this — each chunk shares a few tokens with its neighbour, so no boundary loses context entirely:

```
tokens:   t1  t2  t3  t4  t5  t6  t7  t8  t9  t10 t11 t12

chunk 1: [t1  t2  t3  t4  t5  t6]
chunk 2:             [t5  t6  t7  t8  t9  t10]
chunk 3:                         [t9  t10 t11 t12]
                      ^^^^^^          ^^^^^^
                      overlap         overlap
```

7.5. Implementation

7.6. What problem are we solving?

Split a report's text into pieces small enough to embed precisely, without severing the exact sentence a future query needs right at a chunk boundary.

7.7. Inputs

- Full report text (a Python string).
- `chunk_tokens`: how many tokens each chunk should hold.
- `overlap_tokens`: how many tokens each chunk repeats from the end of the previous one.

7.8. Expected Output

A list of text chunks, each roughly `chunk_tokens` tokens long, each overlapping the next by `overlap_tokens` tokens.

```python
# code/chapter_07/token_chunking.py
import tiktoken

def chunk_text_by_tokens(text: str, chunk_tokens: int = 200,
                          overlap_tokens: int = 40) -> list[str]:
    enc = tiktoken.get_encoding("cl100k_base")
    tokens = enc.encode(text.strip())
    chunks, start = [], 0
    while start < len(tokens):
        end = min(len(tokens), start + chunk_tokens)
        chunks.append(enc.decode(tokens[start:end]).strip())
        if end == len(tokens):
            break
        start = max(0, end - overlap_tokens)
```

```
    return chunks
```

7.9. What just happened?

The text got converted into tokens, then sliced into fixed-size windows that each step forward by less than a full window's width — that gap is the overlap, and it's what keeps a sentence sitting near a boundary from being fully lost to one side or the other.

This is a direct simplification of `chunk_text_by_tokens()` in the companion repository — same sliding-window-with-overlap logic, without the word-based fallback path. Read the full version, plus `split_text_for_segment_aware_chunking_with_patterns()`, in `src/rag_pdf/chunking.py`.

7.10. Production Reality

This chapter's Field Notes (below) shows the heading heuristic getting a perfect score — because Utah FORGE's reports all come from one piece of software, using one consistent template. A larger, multi-operator archive rarely offers that luxury:

- different reporting software formats section headers differently — all-caps, title case, bolded, or not visually marked at all — so a heading heuristic tuned to one archive's convention may need retuning, or replacing entirely, for another
- OCR'd pages (Chapter 6) can corrupt exactly the visual cues a heading heuristic depends on — a garbled, partly-misread line may no longer look like a clean heading at all
- `tiktoken`'s `cl100k_base` encoding matches a specific family of language models; a different embedding or generation model in your pipeline may tokenize the same text differently, so `chunk_tokens=200` isn't automatically the same size everywhere
- overlap improves retrieval at chunk boundaries, but it also multiplies how much text you store and embed — worth measuring deliberately once an archive holds thousands of reports instead of ten

7.11. Practical exercise

Try it yourself: Chunk report #38's full text with `chunk_tokens=60, overlap_tokens=15`, and confirm the stuck-pipe sentence ("During the slide lost tool face and became assembly became stuck") appears complete within at least one chunk rather than split across two.

You'll know it worked when: you can point to a single chunk that contains the full sentence, from "During the slide" through "became stuck."

7.12. Field notes

Field notes: does the heading heuristic ever guess wrong?

Action: run the "all-caps line under 6 words = heading" rule against every line of all ten real reports, and list every match.
Result: every single match — across all ten reports — is a genuine section header: `BHA`, `BOP`, `CASING`, `CONSUMABLES`, `DRILL BITS`, `MUD`, `PUMPS`, `SURVEY DATA`, `TIME BREAKDOWN`, `WELL/JOB INFORMATION`, plus the completion report's `FLUID DATA`, `FRAC`, `PERFORATIONS`, `SAFETY`, and `TIME LOG`. Zero false positives — no data row anywhere in the archive happens to look like a heading.
Why: this isn't luck — it reflects something real about how DDR software generates these reports. Section headers are short, fixed, all-caps labels from a small controlled vocabulary; data rows always carry numbers, units, or lowercase narrative text that the pattern correctly excludes.
Lesson: a heuristic that "seems reasonable" is still a guess until you've run it against real data and checked every match by hand. This one happens to be exactly right on this archive — but that's a claim worth verifying yourself before trusting it on a different one, not an assumption to inherit.

7.13. Challenge exercise

Challenge: Implement a minimal segment-aware splitter that treats any all-caps line under 6 words (like `MUD`, `DRILL BITS`, `SURVEY DATA`, `TIME BREAKDOWN`) as a section heading and starts a new segment there. Run it against report #38's full text and confirm it produces one segment per section rather than one blended mass of casing, mud, and time-breakdown text. A reference solution — and the full production logic — is in `code/chapter_07/challenge/` and `DDR_UTAH_FORGE/src/rag_pdf/chunking.py` respectively.

7.14. Key takeaways

- Chunk size belongs in tokens, because that's the unit every downstream model actually consumes.
- Overlap prevents context from vanishing at a chunk boundary, at the cost of some redundancy in the index.
- Segment-aware boundaries produce chunks that are more likely to be self-contained, retrievable units — worth the extra complexity once reports have real internal structure, which every DDR in this archive does.

7.15. Repository files

File	Purpose
`code/chapter_07/token_chunking.py`	Simplified token-based chunker
`DDR_UTAH_FORGE/src/rag_pdf/chunking.py`	Production token + segment-aware chunking (companion repo)

7.16. What can you do now that you couldn't do before?

You can split a report into chunks small enough to embed precisely, without cutting the exact sentence you need to retrieve in half at a chunk boundary.

7.17. Suggested next step

Well-formed chunks still need somewhere fast to live. Chapter 8 replaces Chapter 4's brute-force cosine search with a real vector database, and scales retrieval from ten documents to the full 76-report archive.

8. Vector Databases at Scale

```
10 Documents
   ↓
76-Report Archive
   ↓
Persistent Index
```

8.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why brute-force cosine search stops being adequate as a corpus grows, in concrete terms (memory and latency).
- Build and query a FAISS index over DDR chunk embeddings.
- Understand the difference between a per-document index and a campaign-wide global index, and when each is the right tool.

8.2. Operational Problem

What happens when your ten-document prototype meets the full archive? Chapter 4's `embeddings @ query_vec` brute-force search works fine over ten documents — it's a single matrix multiply. The full Utah FORGE archive is 76 reports. Run this book's companion pipeline's chunking and embedding steps against all of them and you get **2,943 chunks** — still small by industrial standards, but already the point where you want a persistent, queryable index rather than an in-memory NumPy array you rebuild every session. And a production system needs more than one query mode: per-document search (fast, scoped to one report) and campaign-wide search across every chunk are genuinely different operations.

8.3. Theory

A **vector database** (or vector index library, like FAISS (Johnson et al. 2019)) organises embeddings so that similarity search doesn't require comparing the query against every single vector.

> Engineering Translation: FAISS index
>
> Think of Chapter 4's brute-force search as walking every aisle of a warehouse to find a part. A **FAISS index** is that same warehouse with an addressing scheme: it can go straight to

> the right shelf instead of checking every one. At this book's scale the "shelf" it finds is still mathematically the exact same answer — it just gets there faster and without needing everything loaded into memory by hand each time.

At this book's scale, even the simplest FAISS index — `IndexFlatIP` (flat, inner-product) — is the right choice: it's an *exact* nearest-neighbour search, mathematically equivalent to Chapter 4's brute-force approach, just implemented in optimised C++ with a persistent, loadable index file. (Larger-scale approximate indexes — IVF, HNSW — trade a small amount of accuracy for speed at millions of vectors; not a tradeoff this book's scale needs.)

Because embeddings are normalized (Chapter 4), inner product *is* cosine similarity — which is exactly why `IndexFlatIP` is the right index type rather than a distance-based one.

> Engineering Translation: Persisting an index
>
> **Persisting** an index means saving it to a file, the same way you'd file a completed report in a binder instead of reconstructing it from memory every morning. Build it once, save it, and every future session just loads the finished file — no re-embedding, no rebuilding.

8.4. Building the real index

The companion pipeline, **DDR_UTAH_FORGE**, builds two tiers of index:

- **Per-document index** (`scripts/build_index.py`) — one FAISS index per DDR, used when a question is scoped to a specific report. All 76 reports already have this: chunks, embeddings, and a `faiss.index` file sit in `data/processed/<doc_id>/` for every report in the archive.
- **Global index** (`scripts/build_global_index.py`) — a single FAISS index over every chunk across every report, used for cross-document questions like "which reports involve fishing operations."

Running `build_global_index.py` against this archive's 76 pre-processed documents produces a real, verifiable result:

```
Found 76 docs with embeddings
Concatenating 2,943 chunks from 76 docs...
Building FAISS IndexFlatIP ((2943, 384))...
  faiss.index - 2,943 vectors × 384d
  elapsed: 2.65s
```

384 dimensions matches `all-MiniLM-L6-v2`, the same embedding model from Chapter 4 — this pipeline doesn't switch models between the toy version and the production one, it just changes how the resulting vectors are indexed and queried.

The two tiers branch from the same per-document work, then serve different questions:

```
76 DDR PDFs
   ↓
chunks + embeddings
(per report)
   ↓
built into two indexes:
   ↓
Per-doc FAISS index
(one per report)
-> search THIS report
   ↓
Global FAISS index
(2,943 chunks, all reports)
-> search ACROSS reports
```

8.5. Implementation

8.5.1. Step 1: build the index and save it to disk

8.6. What problem are we solving?

Turn a matrix of chunk embeddings into a fast, queryable structure, then save it so it never needs rebuilding from scratch in a future session.

8.7. Inputs

- `embeddings`: a NumPy array of chunk vectors, shape `(n_chunks, 384)`.
- A destination file path for the saved index.

8.8. Expected Output

A `faiss.index` file on disk, plus the same in-memory index object ready to query immediately.

```python
# code/chapter_08/build_faiss_index.py
from pathlib import Path

import faiss
import numpy as np

def build_index(embeddings: np.ndarray) -> faiss.IndexFlatIP:
    dimension = embeddings.shape[1]
    index = faiss.IndexFlatIP(dimension)
    index.add(embeddings.astype("float32"))
```

```
    return index

def save_index(index: faiss.Index, path: Path) -> None:
    faiss.write_index(index, str(path))
```

8.9. What just happened?

`build_index` creates an empty FAISS structure sized for 384-number vectors, then loads every chunk's embedding into it in one call. `save_index` writes that structure out to a single file — the "binder" from the Engineering Translation above — so the next session can load it back instantly instead of re-embedding every chunk again.

8.9.1. Step 2: reload the index and answer a query

8.10. What problem are we solving?

In a fresh session — no re-embedding, no rebuilding — load the saved index back and answer a query against it, the same way Chapter 4's `search` did, just backed by FAISS instead of a NumPy array in memory.

8.11. Inputs

- The saved `faiss.index` file path.
- A query vector (from Chapter 4's embedding model).
- `top_k`: how many results to return.

8.12. Expected Output

A ranked list of `(chunk_index, similarity_score)` pairs — the same shape of result Chapter 4's brute-force `search` returned.

```
def load_index(path: Path) -> faiss.Index:
    return faiss.read_index(str(path))

def search(index: faiss.Index, query_vec: np.ndarray, top_k: int = 5):
    scores, indices = index.search(query_vec.reshape(1, -1).astype("float32"),
↪   top_k)
    return list(zip(indices[0].tolist(), scores[0].tolist()))
```

8.13. What just happened?

`load_index` reads the saved file back into a ready-to-query FAISS index — no re-embedding step, because the vectors were already saved inside it. `search` hands it one query vector and asks for the `top_k` closest matches; FAISS returns their positions and scores, which get paired up into the same kind of ranked list Chapter 4 produced by hand.

This mirrors the indexing pattern in the companion pipeline's `src/rag_pdf/services/search_service.py`, which builds `IndexFlatIP` over chunk embeddings and persists it to `faiss.index` alongside each document's other artefacts.

8.14. Production Reality

This chapter's `IndexFlatIP` scales cleanly to Utah FORGE's 2,943 chunks — both to build and to query. A single operator's *entire* well portfolio, or a service company handling multiple clients, changes the picture:

- millions of chunks make exact search too slow; that's when approximate indexes (IVF, HNSW) become the right tradeoff, not just an option mentioned in passing
- a flat index file rebuilt from scratch every time a new DDR arrives doesn't scale — production systems need to add new vectors incrementally, without reprocessing the whole archive
- multiple engineers querying the same index at once, from different applications, usually means a managed vector database (pgvector, Pinecone, Weaviate) instead of a single `faiss.index` file on one machine's disk
- if some wells' data is more sensitive than others (a joint-venture well, say, versus a wholly-owned one), a single global index needs an access-control layer on top of it — FAISS itself has no concept of "who's allowed to see this chunk"

8.15. Practical exercise

Try it yourself: Build a FAISS index over the ten sample DDR embeddings from Chapter 4, save it to disk, reload it in a fresh Python session, and confirm a query for "stuck pipe" returns the same ranked order as Chapter 4's brute-force search — report #38 at rank 2, report #39 at rank 5.

You'll know it worked when: the FAISS-based and brute-force results agree — because, mathematically, `IndexFlatIP` over normalized vectors *is* cosine similarity, just implemented differently.

8.16. Field notes

> Field notes: proving 'exact, not approximate' rather than asserting it
>
> **Action:** run five different real queries — `"stuck pipe"`, `"fishing operation"`, `"packers failed to set"`, `"torque problems"`, `"losses"` — through both the brute-force NumPy search and a FAISS `IndexFlatIP` built over the same ten embeddings, and compare the full ranked order each one returns.
> **Result:** all five queries produce byte-for-byte identical rankings between the two methods.
>
> ```
> 'stuck pipe':              brute-force order == FAISS order: True
> 'fishing operation':       brute-force order == FAISS order: True
> 'packers failed to set':   brute-force order == FAISS order: True
> 'torque problems':         brute-force order == FAISS order: True
> 'losses':                  brute-force order == FAISS order: True
> ```
>
> **Why:** this is exactly what the Theory section claimed — `IndexFlatIP` computes the *same* inner-product comparison as `embeddings @ query_vec`, just in optimised C++ rather than NumPy. There's no approximation step anywhere in this index type for it to disagree with brute force on.
> **Lesson:** "mathematically equivalent" is a claim worth testing, not just trusting because it sounds right — especially before you rely on it at a scale where you can no longer eyeball the difference. It took five lines of code to confirm this one.

8.17. Challenge exercise

Challenge: If you have the full 76-report Utah FORGE archive (`datasets/forge_archive/`) and can run the heavier dependencies (`sentence-transformers`, `faiss-cpu`), chunk and embed all 76 reports yourself and confirm you land close to the real 2,943-chunk count above. Small differences are expected — your chunking parameters won't be identical to the production pipeline's — but you should be in the same order of magnitude. A reference solution is in `code/chapter_08/challenge/`.

8.18. Key takeaways

- `IndexFlatIP` at this book's scale is exact, not approximate — you're not trading away accuracy, only reimplementing brute-force search more efficiently and persistently.
- Per-document and global indexes serve genuinely different questions; build both rather than forcing one index to do both jobs.
- Scaling the *index* doesn't fix retrieval *quality* — that's the next chapter's problem.

8.19. Repository files

File	Purpose
`code/chapter_08/build_faiss_index.py`	FAISS `IndexFlatIP` build/save/load/search
`code/chapter_08/build_full_archive.py`	Reproduces the full 76-report archive from the public source
`datasets/forge_archive/`	The full 76-report Utah FORGE archive
`DDR_UTAH_FORGE/scripts/build_index.py`	Per-document index builder (companion repo)
`DDR_UTAH_FORGE/scripts/build_global_index.py`	Campaign-wide global index builder (companion repo)

8.20. What can you do now that you couldn't do before?

You can build a persistent, queryable index over an entire report archive that survives a restart — and you've verified, not just assumed, that scaling up the infrastructure didn't change a single answer the retrieval gives back.

8.21. Suggested next step

A fast index doesn't guarantee good retrieval. Chapter 9 looks at the retrieval scoring code itself — how sparse and dense signals get combined, and what it takes to make that combination numerically sound.

9. Hybrid Retrieval and Reranking

```
Keyword Match + Meaning Match
        ↓
    Fused Ranking
        ↓
  Reranked Results
```

9.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why semantic search alone underperforms on precise, entity- or number-heavy queries.
- Combine BM25 (sparse/exact) and dense (semantic) retrieval using Reciprocal Rank Fusion (RRF).
- Apply a cross-encoder to rerank a small candidate set for higher precision than either retrieval method alone.
- Read production retrieval-scoring code closely enough to explain *why* a specific formula is written the way it is, not just what it computes.

9.2. Operational Problem

Which report had packers fail to set — and why doesn't either search method alone find it reliably? A query like *"which report had packers fail to set?"* has both a semantic component (packers, setting, failure — a topic) and effectively an exact-match component (you want report #49 specifically, not every report that mentions packers in passing). Dense embeddings capture the topical meaning well but blur precise distinctions; keyword search (Chapter 3) nails the exact terms but misses paraphrase entirely. Neither alone is enough — production retrieval systems combine both.

9.3. Theory

BM25 (Robertson and Zaragoza 2009) scores a document by term frequency, adjusted by how rare each term is across the corpus (inverse document frequency, IDF) and normalised for document length. It's decades old, computationally cheap, and excellent at exact and near-exact matches — the opposite profile from dense embeddings.

> Engineering Translation: BM25
>
> **BM25** is keyword search's more careful cousin. It still matches exact words, like Chapter 3's search, but gives more credit for rare, specific terms ("packers") than common ones ("the"), and adjusts for how long each report is so a longer report doesn't win just by containing more words overall.

Reciprocal Rank Fusion (RRF) combines two ranked lists by *rank*, not raw score: for each document, sum `weight / (k + rank)` across both lists. This sidesteps a real problem — BM25 scores and cosine similarities live on completely different, incomparable scales, so averaging raw scores directly would let whichever signal happens to have larger numbers dominate regardless of which is actually more informative.

> Engineering Translation: Reciprocal Rank Fusion
>
> Think of two judges ranking the same horse race. One scores out of 10, the other out of 100 — their raw scores can't be averaged meaningfully. But their *placings* (1st, 2nd, 3rd…) mean the same thing regardless of scale. **RRF** combines two search results the same way: by where each report placed in each ranking, not by the raw number each method happened to produce.

```
Query
   ↓
scored two ways:
   ↓
BM25 (sparse, exact terms)
   ↓
Dense (semantic embedding)
   ↓
Reciprocal Rank Fusion
(combine by rank, not score)
   ↓
Cross-encoder rerank
(top candidates only)
   ↓
Results
```

Cross-encoder reranking is the last, most expensive step, and deliberately runs on only the top handful of fused candidates, not the whole corpus. Unlike a dense embedding (which encodes the query and each document *separately*, then compares vectors), a cross-encoder scores the query and a candidate passage *together*, letting it capture interactions between them that no fixed vector representation can. This makes it substantially slower per comparison — which is exactly why it's applied after fusion has already narrowed the field, not instead of it.

> Engineering Translation: Cross-encoder
>
> A dense embedding is like summarizing the query and each report separately, then comparing the two summaries from a distance. A **cross-encoder** instead reads the query and one

candidate passage *side by side, together*, and judges the pair directly — closer to how a person actually double-checks a match, but too slow to do for every report, which is why it only runs on the small shortlist fusion already produced.

9.4. Reading the real fusion code

The companion pipeline's `src/rag_pdf/retrieval/hybrid_utils.py` and `src/rag_pdf/services/search_service.py` implement exactly this pipeline. Two details in the real code are worth understanding closely, because they're the kind of thing that looks like a stylistic choice until you see what breaks without it.

Detail 1: the IDF formula's shape isn't arbitrary

In plain terms: this line makes sure a very common word can never make a report's score go *negative*. Here's the code:

```
self.idf[term] = math.log(1.0 + ((self.n_docs - df + 0.5) / (df + 0.5)))
```

Compare this to the classic Robertson-Sparse-Selection IDF, `log((N - df + 0.5) / (df + 0.5))` *without* the `1.0 +`. For a term that appears in almost every document (`df` close to `n_docs`), the classic formula's ratio drops below 1 and the log goes **negative** — which then corrupts any downstream sum that assumes term contributions are non-negative. Wrapping the ratio as `1.0 + ratio` before taking the log guarantees the result stays non-negative for every possible `df`, because `1.0 + ratio` can never fall below 1. This single design choice removes an entire category of edge case, permanently, for a cost of one added constant.

Detail 2: normalization has to define its own tie-break

In plain terms: this guards against a division that would otherwise break the whole calculation whenever every candidate ties. Here's the code:

```
if hi <= lo:
    return {i: 0.0 for i in candidates}
```

Min-max normalization rescales scores into `[0, 1]` by subtracting the minimum and dividing by the range. If every candidate in a batch scores identically, `hi - lo` is zero — a division that, left unguarded, would raise or produce `NaN`. The guard above catches that case explicitly and returns a neutral value (0.0 for everyone) instead, so a tie in *this* signal doesn't crash the pipeline or corrupt the fused score — the other signal in the fusion is still free to discriminate between the tied candidates.

Neither of these edge cases shows up by running "more queries" against typical text — they only show up by reasoning about the math at its boundaries: what happens when a term is universal, what happens when every score ties. Retrieval scoring is numerical code, and it deserves the same edge-case discipline you'd apply to any other numerical code. The fusion weights themselves are also plain, readable constants in `search_service.py`: `RRF_K = 20`, `RRF_DENSE_WEIGHT = 0.5`, `RRF_BM25_WEIGHT = 2.0` — the exact-match signal is weighted four times higher than the semantic one in this pipeline's fused ranking, reflecting that for DDR text, exact-term matching often carries more precision than pure similarity.

9.5. Implementation

9.6. What problem are we solving?

Combine two independently-ranked lists of reports — one from BM25, one from dense/semantic search — into a single fused ranking, without letting whichever method happens to produce larger raw numbers unfairly dominate the result.

9.7. Inputs

- `ranked_lists`: two or more ranked lists of document IDs, best match first (e.g. one from Chapter 3's keyword search, one from Chapter 4's semantic search).
- `k`: a constant controlling how much rank position matters.
- `weights`: how much to trust each list relative to the others.

9.8. Expected Output

A single fused ranking: a list of `(doc_id, score)` pairs, sorted highest-scoring first.

```python
# code/chapter_09/hybrid_search.py
from collections import defaultdict

def reciprocal_rank_fusion(ranked_lists: list[list[str]], k: int = 20,
                           weights: list[float] | None = None) ->
↪  list[tuple[str, float]]:
    weights = weights or [1.0] * len(ranked_lists)
    scores: dict[str, float] = defaultdict(float)
    for ranked_list, weight in zip(ranked_lists, weights):
        for rank, doc_id in enumerate(ranked_list, start=1):
            scores[doc_id] += weight * (1.0 / (k + rank))
    return sorted(scores.items(), key=lambda item: item[1], reverse=True)
```

9.9. What just happened?

For every report, this adds up a small credit from each ranked list based on *where it placed* — a high rank (near the top) contributes a bigger credit than a low rank, scaled by that list's weight — then sorts every report by its total credit. Because this uses each list's rank, not its own raw numbers, a method that happens to score everything on a bigger scale can't unfairly take over the fused result.

9.10. Production Reality

This chapter tunes two signals with fixed weights (BM25 × 2.0, dense × 0.5) chosen for DDR text specifically. Real systems have to keep that tuning honest over time, not treat it as a one-time decision:

- these weights were tuned for this domain's language — a different report type (well completions versus drilling, say, or a different operator's writing style) may need genuinely different weights, not the same ones reused out of habit
- cross-encoder reranking adds real per-query latency, the same way an LLM call does (Chapter 5) — it has to be budgeted for, not just switched on and forgotten
- production systems often fuse more than two signals — metadata filters like well name, date range, or report type frequently join BM25 and dense scores in the same fusion step, and each added signal is one more way the weighting can go wrong
- as this chapter's own Field Notes show below, adding a signal isn't automatically an improvement — a production system needs ongoing evaluation (Chapter 11), not a one-time tuning pass assumed to hold forever

9.11. Practical exercise

Try it yourself: Rank the ten sample DDRs for "packers fishing" using Chapter 3's keyword search and Chapter 4's semantic search separately, then fuse the two ranked lists with `reciprocal_rank_fusion(lists, k=20, weights=[2.0, 0.5])` — matching the companion pipeline's real weighting.

You'll know it worked when: report #49 (packers) and #50 (fishing) both rank near the top of the fused list, and you can explain why the weighting favours the keyword signal here.

9.12. Field notes

⚠ Field notes: hybrid fusion isn't automatically better — check both directions

Query: the same `"stuck pipe"` query from Chapters 3 and 4.
Result: with the pipeline's real fusion weights (BM25 × 2.0, dense × 0.5), report #39 ranks **9th of 10** — *worse* than Chapter 4's dense-only search, which ranked it 5th. Equal weighting

(1.0 / 1.0) still only gets it to 7th.

```
BM25 alone:    report #39 ranks 9th of 10
Dense alone:   report #39 ranks 5th of 10   (Chapter 4)
RRF (2.0/0.5): report #39 ranks 9th of 10   (pipeline default weights)
RRF (1.0/1.0): report #39 ranks 7th of 10
```

Compare that to `"packers fishing"` (the practical exercise above), where fusion helps exactly as expected — report #49 stays 1st and report #50 jumps from 8th (dense alone) to 2nd (fused).
Why: BM25 has essentially nothing useful to say about report #39 for `"stuck pipe"` — it shares almost no vocabulary with the query, so BM25 ranks it 9th on its own. Fusing that weak, near-random signal in with a weight of `2.0` doesn't cancel out — it actively drags a mediocre dense result down further. For `"packers fishing"`, by contrast, both signals carry real information (the word "fishing" genuinely appears in report #50), so combining them helps.
Lesson: hybrid retrieval's benefit depends on *both* signals being informative for the query at hand — not on combining more signals being unconditionally better. A sparse signal with nothing to contribute isn't neutral when fused; it's noise with a weight attached, and can make a weak dense ranking worse. This is exactly why Chapter 11 insists on per-category evaluation rather than one aggregate score — "hybrid beats dense on average" can still be actively wrong for a specific, operationally important query like this one.

9.13. Challenge exercise

Challenge: Implement the two guards from the code study above against deliberately constructed edge cases: a term that appears in every sample document (test that your IDF stays non-negative), and a batch where every candidate scores identically (test that your normalization doesn't divide by zero). Confirm an unguarded version fails first. A reference solution is in `code/chapter_09/challenge/`.

9.14. Key takeaways

- Sparse and dense retrieval fail on different, complementary query types — combine them rather than picking one.
- Rank-based fusion (RRF) avoids the scale-mismatch problem that plagues naive score-averaging across different retrieval methods.
- Cross-encoder reranking buys precision at a real latency cost — apply it only to a small, already-fused candidate set.
- Retrieval scoring is numerical code. Boundary conditions — universal terms, tied scores — deserve explicit guards, and reading *why* a formula is shaped the way it is tells you more than reading what it computes.

9.15. Repository files

File	Purpose
`code/chapter_09/hybrid_search.py`	Reciprocal Rank Fusion implementation
`DDR_UTAH_FORGE/src/rag_pdf/retrieval/hybrid_utils.py`	BM25 index, IDF formula, score fusion (companion repo)
`DDR_UTAH_FORGE/src/rag_pdf/retrieval/canonical_hybrid.py`	Full hybrid retrieval pipeline (companion repo)
`DDR_UTAH_FORGE/src/rag_pdf/services/search_service.py`	Cross-encoder rerank + fusion weights (companion repo)

9.16. What can you do now that you couldn't do before?

You can combine exact keyword matching and semantic search into a single ranked list that plays to each method's strengths — and, from real evidence rather than assumption, you know that combining signals only helps when both of them actually have something useful to say about the query at hand.

9.17. Suggested next step

Better retrieval still isn't the same as a trustworthy answer. Chapter 10 tackles the harder problem: making sure every generated claim traces back to real evidence, and that the system tells you honestly when it doesn't know.

10. Traceable Answers and Hallucination Mitigation

```
Plausible Answer
      ↓
Provable Answer
```

10.1. Learning objectives

By the end of this chapter, you will be able to:

- Distinguish questions that should be answered by generation from questions that should be answered by structured data lookup.
- Attach a verifiable citation (report number, page) to every claim a system produces.
- Detect and surface gaps in your own source archive, so the system's silence about a period isn't mistaken for "nothing happened."

10.2. Operational Problem

Can you actually prove that number, or did the model just make it sound right? Chapter 5 built a prompt that *asks* a language model to cite sources — necessary, but not sufficient. A model can still cite a plausible-sounding report number for a number it invented, especially for arithmetic questions ("how many hours of non-productive time in November?") that language models are statistically bad at answering exactly, no matter how good the prompt is. Separately, this archive — like any real archive — has a genuine gap, and a system that answers confidently from what it *has*, without flagging what it's *missing*, can quietly mislead an engineer who assumes silence means nothing happened.

10.3. Structured facts, already extracted

Language models can produce fluent, plausible claims that are simply false — a well-documented failure mode called hallucination (Ji et al. 2023).

> Engineering Translation: Hallucination
>
> A **hallucination** is a confident wrong answer — the language-model equivalent of a new hire who guesses a number rather than admitting "I don't know," and says it fluently enough that

> nobody thinks to double-check. It isn't lying on purpose; it's producing text that sounds right without anything underneath actually verifying it.

The single most effective mitigation in this book is a rule of thumb, not a model trick: **if a question can be answered by looking something up or computing it from structured data, don't generate the answer — compute it.**

Every report in this archive has already been parsed into a real, structured `ddr_facts.parquet` table by the companion pipeline. Here's an actual row from report #38 (2020-11-26):

> **A real row from ddr_facts.parquet, report #38**
>
> ```
> report_date: 2020-11-26        wellbore: FORGE-16A-78-32
> start_time:  06:00              end_time:      11:00
> duration_hr: 5.0                 phase:         Production Drilling
> op_code:     Wire Line Logs      is_npt:        False
> operation_text: "PJSM, pre job safety meeting. Rig up Schlumberger
>                  run the UBI log from 5.200 to surface. Rig down
>                  logging tools."
> ```

A question like *"how many hours did report #38 spend on wireline logging?"* has an exact answer sitting in this table — `duration_hr = 5.0` — computed once during extraction, not re-derived by a language model each time it's asked. Generation is reserved for what it's actually good at — narrative synthesis across multiple passages — and even then, every claim carries its source.

```
question
   ↓
answerable from structured data?
   ↓
yes -> query ddr_facts.parquet
    -> exact, re-derivable answer
   ↓
no  -> retrieve + generate (Ch.5)
    -> answer + citations
       (verify before trusting)
```

10.4. A real, honest gap in this archive

The second mitigation is **corpus-completeness awareness**. This archive has a genuine one: the last Drilling report is #76, dated **2021-01-03**. The Completion report (a DFIT — Diagnostic Fracture Injection Test) is dated **2021-01-06**. Two calendar days — January 4th and 5th — have no report of any kind. Every other day in the drilling programme, from spud on October 30, 2020 through report #76, has exactly one report, so this gap is real and specific to the Drilling-to-Completion handover, not an artefact of messy source data.

A RAG system only knows what's in its index. If you ask it "what happened on January 5th, 2021?" without gap-awareness, a poorly built system might either hallucinate an answer or simply retrieve the nearest chunks (report #76 or the Completion report) without ever telling you neither actually covers that date. Detecting the gap is a matter of checking the *sequence* of report numbers and dates for holes — not the content, just the presence.

10.5. Implementation

10.5.1. Step 1: give every claim a checkable tag

10.6. What problem are we solving?

Attach a checkable source to every claim — which report, which page — instead of a citation-shaped sentence a reader has to take on faith.

10.7. Inputs

- A report name/number, and an optional page number.
- A list of such citations to attach to one answer.

10.8. Expected Output

A plain-text "Evidence:" block listing each source, ready to show alongside a generated answer.

```
# code/chapter_10/traceable_answers.py
from dataclasses import dataclass

@dataclass
class Citation:
    report: str
    page: int | None = None

def format_evidence(citations: list[Citation]) -> str:
    lines = [f"  {c.report}" + (f" page {c.page}" if c.page else "") for c in
 ↪  citations]
    return "Evidence:\n" + "\n".join(lines)
```

10.9. What just happened?

`Citation` bundles a report and a page number together as one unit — the equivalent of an evidence tag on an exhibit, so a claim's source can never get separated from the claim itself by accident. `format_evidence` turns a list of those tags into a plain block of text a reader can check against the real reports, one line per source.

> Engineering Translation: Dataclass
>
> A **dataclass** like `Citation` is a small, labelled bundle of related facts — the code equivalent of an equipment tag that keeps a part number and its location together on one card, instead of as two separate loose numbers you could mix up.

10.9.1. Step 2: check the archive for real gaps

10.10. What problem are we solving?

Detect whether the archive has days with no report at all, so the system can say "I have nothing for that date" instead of guessing or silently retrieving the nearest available report.

10.11. Inputs

- A list of report dates, as ISO date strings, covering a period that should have one report per day.

10.12. Expected Output

A list of `(start, end)` date ranges with no report — empty if the archive is complete.

```python
def find_date_gaps(report_dates: list[str]) -> list[tuple[str, str]]:
    """Return (start, end) date ranges with no report, given a sorted
    list of ISO date strings covering a continuous daily-reporting period."""
    from datetime import date, timedelta

    dates = sorted(date.fromisoformat(d) for d in report_dates)
    gaps = []
    for prev, curr in zip(dates, dates[1:]):
        missing_days = (curr - prev).days - 1
        if missing_days > 0:
            gaps.append(((prev + timedelta(days=1)).isoformat(), (curr -
↪   timedelta(days=1)).isoformat()))
    return gaps
```

10.13. What just happened?

The dates get sorted, then checked one consecutive pair at a time: if more than one day passes between two reports, everything in between gets recorded as a missing range. No content is inspected here at all — this only checks whether a report *exists* for each day, which is exactly enough to catch the real two-day gap in this archive.

10.14. Production Reality

This chapter's citation and gap-detection code both assume the underlying extraction is trustworthy — but structured extraction is still code, written by someone, and can encode a judgment call worth checking (see this chapter's own Field Notes on `is_npt` below). A deployed system has more to plan for:

- gap detection has to run continuously as new reports arrive, not just once at setup — a system that only checked for gaps on day one won't notice a new one appearing three months later
- not every gap is administrative silence — a missing day could mean a missing safety report, which is worth escalating to a person, not just logging quietly as "no data for this date"
- a citation is only useful if someone actually reads it — a system that buries the evidence list at the bottom of a long answer, in small print, earns the same blind trust as a system with no citations at all
- some engineering decisions (regulatory submissions, incident reports) require a human sign-off no matter how good the system's citations are — traceability supports a human reviewer, it doesn't replace one

10.15. Practical exercise

Try it yourself: Run `find_date_gaps()` against the report dates in this book's ten-report sample — `["2020-10-22", "2020-11-07", "2020-11-24", "2020-11-25", "2020-11-26", "2020-11-27", "2020-12-06", "2020-12-07", "2020-12-08", "2021-01-06"]`.

You'll know it worked when: the function reports several gaps — most of them simply reflecting that Part I's curated subset skips most of the archive's days on purpose. Then run it against the *full* 76-report archive's dates (`datasets/forge_archive/`) and confirm it finds exactly one real gap: January 4th–5th, 2021, right before the Completion report.

10.16. Field notes

⚠ Field notes: verify the automatic label, not just the automatic lookup

Action: check the real, computed `is_npt` flag — set once during extraction, not re-derived on every query — for every operations row on report #38, the stuck-pipe day.
Result: out of ten rows covering the full 24 hours, exactly **one** is flagged `is_npt = True`:

```
23:30-04:00  is_npt=True   "Drill From 6,360' to 6,507'... During the
                            slide lost tool face and became assembly
                            became stuck"
04:00-06:00  is_npt=False  "Work pipe, circulate lube sweep, work tool
                            back in position, Pipe free"
```

Why: the classifier flagged the 4.5-hour block where the pipe *became* stuck, but not the following 2-hour block where the crew actually recovered it. That's a defensible reading — the drilling operation is what got interrupted — but it's also a real judgment call: an engineer

> asking "how many hours did this stuck-pipe event cost?" would probably expect all 6.5 hours counted, not 4.5.
> **Lesson:** a structured field being *automatically extracted* doesn't mean it's automatically *correct for your question.* Trusting `ddr_facts.parquet` blindly here would silently undercount the incident by 2 hours, every time someone asks. The fix isn't to distrust structured data — Chapter 10's whole argument is that it beats generation — it's to read the classification logic once, understand exactly what it counts, and know the edge it sits on before you build a report or a chart on top of it.

10.17. Challenge exercise

Challenge: Extend `find_date_gaps()` to classify each gap's severity (e.g. a gap under 3 days is Low; a gap that crosses a report-type boundary — Drilling to Completion, as this one does — is High regardless of length, since it likely represents an unrecorded operational transition). A reference solution is in `code/chapter_10/challenge/`.

10.18. Key takeaways

- Prefer structured lookup over generation whenever the question is answerable from data you already extracted — it's exact, not just plausible.
- Every generated claim needs a citation an engineer can independently check, not just a citation-shaped sentence.
- A system that can tell you what it *doesn't have* is more trustworthy than one that answers fluently regardless of coverage — this archive's real two-day gap at the Drilling-to-Completion handover is exactly the kind of silence worth surfacing explicitly, not filling in.

10.19. Repository files

File	Purpose
`code/chapter_10/traceable_answers.py`	Citation formatting and date-gap detection
`DDR_UTAH_FORGE/data/processed/qc/raw_pdf_missing_reports.csv`	Real, computed gap-detection output (companion repo)

10.20. What can you do now that you couldn't do before?

You can tell the difference between a question you should compute from data you already have and one you should generate an answer to — and you can prove, with a checkable citation, exactly where every part of a generated answer came from, including telling an engineer honestly when the archive simply has nothing to say.

10.21. Suggested next step

You now have a system that retrieves well and answers honestly. Chapter 11 asks the question every system like this eventually needs answered with numbers, not impressions: *how good is it, really* — and where specifically does it still fail?

11. Evaluating Retrieval Quality

```
"It Feels Like It's Working"
            ↓
Measured, Defensible Performance
```

11.1. Learning objectives

By the end of this chapter, you will be able to:

- Build a hand-labeled evaluation set that records, per question, the expected source report and page.
- Compute `recall@k`, Mean Reciprocal Rank (MRR), and `NDCG@k`, and explain what each number actually tells you.
- Break results down by question category and difficulty to find *where* a system fails, not just *whether* it does.

11.2. Operational Problem

"It feels like it's working" is not a standard you should ship an engineering decision-support tool against. Before this system goes in front of drilling engineers, you need a defensible answer to "how good is it" — and, more usefully, "which kinds of questions does it get wrong." The companion pipeline ships an evaluation harness for exactly this, `scripts/run_eval.py`, built against a real question set for a different (private, North Sea) campaign it was originally developed on. It has **not yet been run against the Utah FORGE archive** — no evaluation question set exists for this well yet. That's not a gap in the tooling; it's the natural next step, and this chapter walks you through doing it for real, against the same archive you've used throughout this book.

11.3. Building your own question set

Rather than presenting borrowed results, build a small evaluation set against the ten-report Part I sample, where you already know the ground truth from having read the reports yourself in Chapters 1–5:

> ℹ A starter evaluation set for the Part I sample
>
> ```
> Q1: "Which report describes pipe becoming stuck during a slide?"
>     expected: FORGE-16A-78-32_Drilling_038_2020-11-26.pdf
> ```

```
Q2: "Which report shows packers failing to set?"
    expected: FORGE-16A-78-32_Drilling_049_2020-12-07.pdf

Q3: "Which report describes milling up lost pieces of bit?"
    expected: FORGE-16A-78-32_Drilling_050_2020-12-08.pdf

Q4: "Which report mentions mud fluid seepage losses?"
    expected: FORGE-16A-78-32_Drilling_019_2020-11-07.pdf

Q5: "Which report shows the crew rigging up before spud?"
    expected: FORGE-16A-78-32_Drilling_003_2020-10-22.pdf
```

Each question has exactly one verifiable correct answer, because you picked these five reports for their distinct, checkable events in Chapters 1–5. This is the right way to start an evaluation set: begin with cases you can verify by eye, before scaling to enough volume that you can no longer read every answer yourself.

11.4. Theory

Three metrics, each answering a slightly different question:

- `recall@k` — did the correct answer appear *anywhere* in the top k results? Simple, and the right first metric to compute.
- **Mean Reciprocal Rank (MRR)** — averages `1 / rank` of the correct answer across all questions. Rewards getting the right answer to rank #1, not just somewhere in the top 10 — closer to what a user actually experiences.
- `NDCG@k` (Normalized Discounted Cumulative Gain) — like recall, but weights a hit at rank 1 more than a hit at rank 5, and normalizes so the score is comparable across queries with different numbers of correct answers.

Engineering Translation: recall@k

`recall@k` asks a plain yes/no question: did the right report make your shortlist of the top `k` results at all? It's like asking whether the correct casing point made your shortlist of candidate depths — it doesn't care whether it was the first one you'd pick or the last, only whether it was on the list.

Engineering Translation: Mean Reciprocal Rank

MRR rewards a correct answer for landing near the top, not just somewhere on the list. It's the difference between "the right depth was in my top 5" and "the right depth was my very first pick" — the second is worth more, and MRR is the metric that actually reflects that.

> **Engineering Translation: NDCG@k**
>
> **NDCG@k** is a more careful version of `recall@k`: instead of a flat yes/no, it gives partial credit based on exactly where the right answer landed — full credit at rank 1, a bit less at rank 2, less still further down — rather than treating "found it at #1" and "found it at #5" as equally good.

None of these require deep statistics to use well — they require a good evaluation *set*: real questions, with a real, known correct source recorded in advance. Building that set by hand, from your own archive, is more valuable than the metric computation itself, because writing the questions forces you to think like the engineer who will actually use the system.

```
hand-labeled questions
(question -> expected report)
   ↓
run each through the retrieval pipeline
   ↓
record the rank of the expected report
   ↓
compute recall@k, MRR, NDCG@k
   ↓
break results down by category and difficulty
   ↓
find the weakest category -> that's where to improve next
```

11.5. Implementation

11.5.1. Step 1: did the right answer make the shortlist at all?

11.6. What problem are we solving?

Check, for every evaluation question, whether the correct report showed up anywhere in the top `k` retrieved results — a simple yes/no per question.

11.7. Inputs

- A list of result dicts, one per evaluation question, each recording the rank the correct report was found at (or `None` if it wasn't found).
- `k`: how far down the results to look.

11.8. Expected Output

A single number between 0 and 1 — the fraction of questions where the correct report was within the top `k`.

```
# code/chapter_11/eval_metrics.py
def recall_at_k(results: list[dict], k: int) -> float:
    hits = [r for r in results if r.get("rank") is not None and r["rank"] <= k]
    return len(hits) / len(results) if results else 0.0
```

11.9. What just happened?

For each question, this checks whether the correct report's rank is within `k` — a plain hit or miss — then reports what fraction of all questions counted as a hit. It says nothing about *how close to the top* a hit was, which is exactly the gap the next two metrics fill.

11.9.1. Step 2: reward landing near the top, not just anywhere

11.10. What problem are we solving?

Give more credit to a correct answer that ranked first than one that merely appeared somewhere in the results — closer to what a user actually experiences when scanning a results list top to bottom.

11.11. Inputs

- The same list of result dicts as Step 1.

11.12. Expected Output

A single average score: higher when correct answers tend to rank near the top, lower when they're buried further down or missing.

```
def mrr(results: list[dict]) -> float:
    scores = [1.0 / r["rank"] if r.get("rank") else 0.0 for r in results]
    return sum(scores) / len(scores) if scores else 0.0
```

11.13. What just happened?

Each question's score is `1 / rank` — rank 1 scores a full 1.0, rank 5 scores only 0.2, and a miss scores 0. Averaging those scores across every question produces one number that rewards *first-place* correct answers far more than answers merely present somewhere in the list.

11.13.1. Step 3: give partial credit based on exact position

11.14. What problem are we solving?

Combine the best of both previous metrics: a graded score, like MRR, but one that treats "found in the top k" as the relevant window, like recall@k.

11.15. Inputs

- The same list of result dicts, plus `k`.

11.16. Expected Output

A single average score, weighted so a hit at rank 1 counts more than a hit at rank 5, and anything beyond `k` counts as zero.

```
def ndcg_at_k(results: list[dict], k: int) -> float:
    import math
    scores = []
    for r in results:
        rank = r.get("rank")
        scores.append(1.0 / math.log2(rank + 1) if rank and rank <= k else 0.0)
    return sum(scores) / len(scores) if scores else 0.0
```

11.17. What just happened?

Instead of `1 / rank` (MRR), each hit's credit shrinks by `1 / log2(rank + 1)` — a gentler slope that still rewards a rank-1 hit the most, still penalizes a rank-5 hit, but doesn't fall off quite as sharply. Anything outside the top `k` scores zero, same as `recall@k`.

Each `result` dict records, per evaluation question, the rank at which the known-correct report was retrieved (or `None` if it wasn't in the top k considered). This mirrors the shape of `_recall_at_k()`, `_mrr()`, and `_ndcg_at_k()` in the companion repository's `scripts/run_eval.py`.

11.18. Production Reality

The evaluation set in this chapter has five questions, and the challenge exercise's has fifteen — genuinely useful, but still small next to a real deployment. A few realities worth planning for:

- an evaluation set needs to keep growing as new report types, wells, and question styles show up — five questions that were exhaustive for the Part I sample won't stay representative of a live system fielding real engineers' questions
- tuning the retrieval pipeline specifically to score well against your own evaluation set risks overfitting to it — the same way memorizing an exam's specific questions isn't the same as knowing the material. Holding some questions back, or continuously adding new ones, keeps the score honest
- an aggregate metric is a summary, not a substitute for periodically reading real answers by eye — especially before a decision with real consequences rides on the system's output
- who writes the evaluation questions matters — a question set written entirely by the person who built the retrieval pipeline tends to test what that person already thought to check, not what a working engineer will actually ask

11.19. Practical exercise

Try it yourself: Run the five questions above through Chapter 9's hybrid search over the ten-report sample archive, record each correct report's rank, and compute `recall@3` by hand.

You'll know it worked when: you have a small CSV or JSON file mapping question → expected report → observed rank, and a `recall@3` score you can quote and defend — because you wrote and verified every question yourself.

11.20. Field notes

! Field notes: 'better' isn't monotonic — track one hard question across every chapter

Action: track report #39's rank for the query `"stuck pipe"` through every retrieval method this book has built so far, instead of just looking at one final aggregate score.
Result:

```
Chapter 3  keyword (AND)         not found at all   (0 of 10)
Chapter 4  dense, whole-document  rank 5 of 10
Chapter 9  hybrid, pipeline weights (BM25x2.0/dense x0.5)  rank 9 of 10
Chapter 9  hybrid, equal weights (1.0/1.0)                 rank 7 of 10
```

Why: each chapter's technique is a genuine improvement *in general* — that's not in question. But this specific, operationally important question doesn't improve monotonically as the pipeline gets more sophisticated. It gets a little better (Chapter 4), then gets worse twice in a row (Chapter 9's two weightings) before hybrid retrieval's other strengths (Chapter 9's `"packers fishing"` example) show up clearly.

Lesson: a single aggregate metric — `recall@5` averaged across your whole evaluation set — can climb steadily from chapter to chapter while a specific, real question you care about gets worse. This is exactly why Chapter 9 insisted on per-category breakdowns and why this chapter opened by building your own question set rather than trusting someone else's summary number: the only way to know if *your* hard questions are improving is to track them individually, the way this table just did.

11.21. Challenge exercise

Challenge: Extend your five-question set to at least 15 questions against the *full* 76-report archive (`datasets/forge_archive/`), and compute `recall@5`. This is genuinely useful, unpublished work: as far as this book's companion pipeline is concerned, you'll be building the first real evaluation set for Utah FORGE. Save your question set alongside your results — it's the seed of exactly the kind of evaluation harness `scripts/run_eval.py` runs at scale once one exists. A reference solution approach is in `code/chapter_11/challenge/`.

Don't be alarmed if `recall@5` comes back much lower than you'd expect from Part I — the reference solution's own 15-question set, run against all 76 reports with Chapter 4's plain whole-document embeddings, scores `recall@5` of only **0.47**. That's not a bug: at 10 candidates, chance alone gets you partway there, and whole-document dilution barely matters. At 76 candidates, both problems compound. A low score here is the honest, measured version of the same lesson Chapter 9 already taught — it's exactly what motivates hybrid retrieval and reranking before this system sees a real, full-scale archive.

11.22. Key takeaways

- An evaluation set with known correct answers is more valuable than any single retrieval technique — it's what tells you whether a technique helped.
- `recall@k`, MRR, and `NDCG@k` answer different questions; report more than one.
- Don't trust a system's evaluation results from a different dataset as a proxy for how it performs on yours — a harness that scored well on one campaign says nothing certain about a new well until you build and run a question set against that well's own archive, which is exactly what this chapter had you do.

11.23. Repository files

File	Purpose
`code/chapter_11/eval_metrics.py`	`recall@k`, MRR, `NDCG@k` implementations
`DDR_UTAH_FORGE/scripts/run_eval.py`	Evaluation harness (companion repo) — ready to point at a Utah FORGE question set once you build one

11.24. What can you do now that you couldn't do before?

You can put a defensible number on how well your retrieval system actually works, broken down by the kinds of questions it handles well and the ones it doesn't — instead of shipping a decision-support tool on the strength of "it feels like it's working."

11.25. Suggested next step

Every discipline from Chapters 6–11 — OCR gating, structured chunking, hybrid retrieval, traceability, evaluation — exists in service of a single payoff: finding things in an archive a human reading it linearly would miss. Chapter 12 builds toward that payoff, using a real sequence of events already in this archive.

12. Sequence Detection: Building Toward Cross-Well Intelligence

```
1 Well, 76 Reports
        ↓
  Checkable Patterns
        ↓
Toward Cross-Well Intelligence
```

12.1. Learning objectives

By the end of this chapter, you will be able to:

- Combine structured numeric trends (torque, ROP) with narrative operation text to build a single, checkable operational story.
- Explain how a windowed leading-indicator detector works — and verify, from real thresholds in production code, exactly what "causal" and "escalating" mean before trusting either label.
- Understand honestly what this book's single-well archive can and can't demonstrate, and what it would take to scale this technique to genuine cross-well intelligence.

12.2. Operational Problem

Did something earlier predict trouble later? This is the payoff question every previous chapter has been building toward. A human reading 76 reports in sequence can, with effort, spot a pattern — but a system that has structured, classified, and indexed the same text can hold every report in mind at once, and check numeric trends a human would have to read a table on every single page to notice.

This chapter is deliberately honest about scale: Utah FORGE's public archive is **one well, one continuous drilling programme** — not the multi-well, multi-phase campaign this book's companion pipeline was originally built to analyse. So instead of presenting a large statistical finding, this chapter shows you exactly how to check a real, small pattern by hand, using the same technique — and shows you precisely what production code exists to scale that technique up once you have more than one well's worth of data.

12.3. A real, checkable pattern

Report #38's `TORQUE` header field — a single number recorded every day regardless of what happens — tells a real story across three consecutive days:

> Real header data, reports #36-38
>
> ```
> Report #36 (2020-11-24): TORQUE: 4,400
> Report #37 (2020-11-25): TORQUE: 4,200
> Report #38 (2020-11-26): TORQUE: 5,615 (header) / 6,500 (during the slide)
>                           "During the slide lost tool face and became
>                            assembly became stuck"
> ```

Torque was flat — even slightly down — for two days, then jumped by roughly a third on the same day the assembly became stuck.

> Engineering Translation: Leading indicator vs. same-day correlation
>
> A **leading indicator** is a signal that shows up *before* the trouble — like a rising torque trend days ahead of an actual stuck-pipe event, which would genuinely give you warning. A **same-day correlation** just means two things moved together on the same day — useful evidence that they're related, but it confirms trouble, it doesn't predict it. This chapter's torque pattern is the second kind, and it matters to be precise about which one you're actually looking at.

This is a same-day correlation between a structured number and a narrative event, not a multi-day advance warning — and that distinction matters. Don't claim a leading indicator you haven't actually verified extends before the event; this pattern says "torque and the stuck-pipe event moved together," not "torque predicted the stuck pipe two days out."

Chapter 5's other real sequence — report #49's packers failing to set, followed same-day by picking up a fishing BHA, followed by report #50's actual fishing operation — is a genuine two-report causal chain, directly traceable and already fully verified with real text.

12.4. Theory: how the production tool would check this at scale

The companion pipeline's `src/ddr_rag/causality_analyzer.py` implements exactly this kind of check, generalized: for a given term, it compares frequency in a window before and after a boundary date, and labels a term **causal** only above a defined post-boundary NPT rate, and **escalating** only above a defined frequency-increase ratio. The real thresholds, read directly from the module:

```
TRANSITION_WINDOW_DAYS = 14   # days before/after a boundary to compare
MIN_TERM_FREQ = 3             # minimum occurrences before a term counts at all
NPT_SIGNAL_THRESHOLD = 0.40   # post-boundary NPT rate to call a term "causal"
ESCALATION_FREQ_RATIO = 1.5   # frequency increase to call a term "escalating"
```

Engineering Translation: Threshold constants

These four lines are named, readable numbers you could point to on a whiteboard and defend in a meeting — "we call it escalating once frequency rises by 50%" — rather than a judgment buried invisibly inside a trained model's weights. Anyone can read them, question them, and change them without touching the logic around them.

Every one of these is a plain, readable number you can inspect, change, and justify — not a weight buried inside a trained model.

! A real limitation worth understanding, not hiding

This module's default phase configuration — `["MIRU", "COND1", "INTRM1", "INTRM2", "PROD1", "COMPZN"]` — comes from `operator_alpha`, a different (private, North Sea) campaign's phase vocabulary. Utah FORGE's real data doesn't have that phase structure: its `ddr_facts.parquet` phase field is `"Production Drilling"` for essentially the entire programme, right up to the Completion-DFIT report. Running `causality_analyzer.py` against this archive as-is would compare windows around phase boundaries that don't meaningfully exist in this well's data — the code would run, but the output wouldn't mean anything useful yet.

This is a genuinely common situation in real engineering work: code built and validated on one dataset doesn't automatically transfer to another just because the file paths line up. Before trusting output from an adapted pipeline, check that the assumptions baked into its configuration (here, a phase vocabulary) actually match your data. The `src/ddr_rag/npt_classifier.py` module, by contrast, *has* been adapted for Utah FORGE specifically — `classify_utah_forge_npt()` exists and recognises this archive's real fields — which tells you adaptation here is partial, not absent: some layers of this pipeline are ready for this well's data, and some aren't yet.

12.5. Implementation

12.5.1. Step 1: measure day-over-day change, not just the raw numbers

12.6. What problem are we solving?

Turn a list of daily torque readings into a measured day-over-day percentage change, so "roughly a third" becomes an exact, checkable number.

12.7. Inputs

- `readings`: a chronological list of `(date, torque_value)` pairs.

12.8. Expected Output

A list of `(date, torque_value, percentage_change)` — one entry for every day after the first, since a percentage change needs a previous day to compare against.

```
# code/chapter_12/torque_trend_check.py
def torque_trend(readings: list[tuple[str, float]]) -> list[tuple[str, float,
↪   float]]:
    """Given [(date, torque_value), ...] in chronological order, return
    each day's percentage change from the previous day."""
    trend = []
    for (prev_date, prev_val), (date, val) in zip(readings, readings[1:]):
        pct_change = (val - prev_val) / prev_val if prev_val else 0.0
        trend.append((date, val, pct_change))
    return trend
```

12.9. What just happened?

This walks through the readings one consecutive pair at a time and computes how much each day's torque changed relative to the day before, as a percentage — the exact arithmetic behind the "flat, then a third higher" story told in prose above.

12.9.1. Step 2: run it against report #38's real numbers

12.10. What problem are we solving?

See the actual percentage change for reports #36–38, printed as real numbers instead of an approximate description.

12.11. Inputs

- The three real `TORQUE` header values from reports #36, #37, and #38.

12.12. Expected Output

```
2020-11-25: 4200 (-4.5%)
2020-11-26: 5615 (+33.7%)
```

```
readings = [
    ("2020-11-24", 4400),
    ("2020-11-25", 4200),
    ("2020-11-26", 5615),
]
for date, val, pct in torque_trend(readings):
    print(f"{date}: {val} ({pct:+.1%})")
```

12.13. What just happened?

Running the real header values through `torque_trend` confirms the pattern precisely: a 4.5% *dip* the day before, then a 33.7% jump on the stuck-pipe day itself — turning "roughly a third" into an exact, reproducible number sourced from the report headers themselves.

12.14. Production Reality

This chapter's torque check works because Utah FORGE's header format is completely consistent, and because a single well gives you exactly one example of the pattern. Genuine cross-well intelligence has different demands entirely:

- different rigs and BHA configurations have different normal torque ranges — a 25% jump threshold tuned on this well could be meaningless, or far too sensitive, on a well drilled with different equipment
- not every operator's DDR software populates a header field as cleanly as WellEz does here — a real multi-operator archive may need per-source parsing before any threshold-based check is even possible
- one well gives you one instance of a pattern, which is a hypothesis, not a finding — genuine cross-well intelligence needs enough wells to tell a real, recurring precursor apart from coincidence, which is exactly why the companion pipeline's `causality_analyzer.py` was built against a many-well campaign in the first place
- as the callout above shows, code built and tuned for one dataset's phase vocabulary doesn't just work on a new dataset because the file paths match — every threshold and configuration assumption needs re-checking against the new data before you trust its output

12.15. Practical exercise

Try it yourself: Extend `readings` with report #39's header torque value (check the PDF yourself — Chapter 1's `read_ddr.py` will get it for you) and confirm whether the elevated torque persisted after the pipe was freed, or dropped back toward the pre-event baseline.

You'll know it worked when: you can state the real percentage change for each day, sourced from header fields you extracted yourself, not from this chapter's text.

12.16. Challenge exercise

Challenge: Pull the `TORQUE` header value for all 76 reports in `datasets/forge_archive/` (Chapter 1's extraction plus a small regex), plot it as a time series, and identify every day where torque jumps more than 25% from the previous day. Cross-reference each jump against that day's narrative text — how many of them correspond to a real operational event like report #38's, and how many are unremarkable? A reference solution is in `code/chapter_12/challenge/`.

12.17. Key takeaways

- A single structured number, tracked day over day, can corroborate a narrative event — but be precise about what you've actually shown: same-day correlation is not the same claim as a multi-day leading indicator, and this chapter deliberately didn't claim more than the real data supports.
- Production causality-detection code carries real, inspectable thresholds — read them before trusting output, and check that its configuration (like a phase vocabulary) actually matches your dataset before running it.
- Genuine cross-well intelligence needs more than one well's worth of data to compare against — Utah FORGE's `data/fields/UtahForge/wells/` structure in the companion pipeline is already organized to support additional wells as this research programme (or your own multi-well archive) grows.

12.18. Repository files

File	Purpose
`code/chapter_12/torque_trend_check.py`	Day-over-day trend check on a structured header field
`DDR_UTAH_FORGE/src/ddr_rag/causality_analyzer.py`	Windowed leading-indicator detection (companion repo)
`DDR_UTAH_FORGE/src/ddr_rag/npt_classifier.py`	Utah-FORGE-adapted NPT classification (companion repo)
`DDR_UTAH_FORGE/data/fields/UtahForge/wells/`	Multi-well data structure, ready for expansion (companion repo)

12.19. What can you do now that you couldn't do before?

You can check, by hand and with real numbers, whether a structured trend and a narrative event actually moved together — and you know precisely what's still missing (more wells, re-tuned thresholds, a matching phase vocabulary) before that check becomes genuine cross-well operational intelligence.

12.20. The complete system

Twelve chapters ago, this was a single PDF and a five-line function. Here's everything you built, end to end — every arrow below is a chapter you completed, not an aspiration:

```
DDR PDF (scanned or digital)
   ↓
OCR quality gate                (Chapter 6)
   ↓
Extraction                      (Chapter 1)
   ↓
Cleaning                        (Chapter 2)
   ↓
Segment-aware chunking          (Chapter 7)
   ↓
Embeddings                      (Chapter 4)
   ↓
Hybrid index - BM25 + FAISS     (Chapters 8-9)
   ↓
Retriever + Reranker            (Chapter 9)
   ↓
LLM + structured facts          (Chapter 5, 10)
   ↓
Answer + Citations + Gaps flagged   (Chapter 10)
```

Two more pieces sit alongside this pipeline rather than inside it — they don't run on every query, but they're what make the rest of it trustworthy at scale:

```
Evaluation (Chapter 11) ---------- measures how well the pipeline above
                                    actually performs, against questions
                                    with known correct answers

Sequence detection (Chapter 12) -- mines what the pipeline has already
                                    indexed for patterns no single query
                                    would surface on its own
```

Every stage exists because an earlier, simpler version of this system failed at it — a scanned page with no text, a chunk that split a stuck-pipe sentence in half, a query that missed a report because it used the wrong three words, a hallucinated number where a table lookup belonged. None of the complexity here is complexity for its own sake.

12.21. Suggested next step

You've now built, chapter by chapter, a working RAG system grounded entirely in real, public DDRs — from extracting a single PDF's text through checking a real numeric trend against a real narrative event. Appendix A shows how to set up the full companion pipeline and point it at your own DDR archive, whether that's more Utah FORGE data as it becomes available, or a different well entirely.

Part III.

Appendices

Appendix A: Environment Setup

Before you can type a single line of Chapter 1's code, your computer needs somewhere safe to install exactly the tools this book uses — without disturbing any other software already on your machine, and without any other software on your machine interfering with this book's code. That's the whole job of this appendix: get that safe, isolated setup in place once, so every chapter from here on is just "open a file and run it."

This appendix covers everything you need before Chapter 1: installing dependencies, rendering the book, and understanding how Chapters 6–12 relate to the companion DDR Intelligence Platform pipeline.

1. Prerequisites

- Python 3.11 or later
- Quarto 1.7 or later (`quarto --version` to check)
- Git
- Positron (recommended) or any editor with a Python and Quarto extension

> Engineering Translation: git clone
>
> This book's entire repository — every chapter's text and code, plus the sample DDR PDFs — lives on GitHub. `git clone <url>` copies that whole repository onto your own computer, once, as a real folder you can open, edit, and run — the same idea as copying a shared project folder onto your laptop instead of only viewing it in a browser.

2. Set up the book's Python environment

From the `book/` directory:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Or, if you prefer conda/mamba:

```
conda env create -f environment.yml
conda activate ddr-rag-book
```

> **Engineering Translation: Virtual environment**
>
> A **virtual environment** (`.venv`) is a clean, separate toolbox that belongs only to this book's project — the software equivalent of a dedicated tool crib kept apart from everyone else's gear. Installing packages inside it can never conflict with, break, or get mixed up with any other Python software already on your computer. `source .venv/bin/activate` is the step that says "use tools from *this* crib for now" — you'll see your terminal prompt change to show it's active.

> **Engineering Translation: pip install -r requirements.txt**
>
> `requirements.txt` is a parts list — one library per line, with the exact version this book was tested against. `pip` is the ordering system: `pip install -r requirements.txt` reads that list and installs every part on it in one go, instead of you tracking down and installing each library by hand.

3. The sample dataset

Chapters 1–5 use ten real, curated Daily Drilling Reports from the public Utah FORGE archive (well FORGE 16A(78)-32) — already committed as PDFs in `datasets/sample_ddrs/`, since this is public data with no confidentiality concerns. Part II's "at scale" chapters use the full 76-report archive, committed at `datasets/forge_archive/`. Nothing needs to be generated or downloaded to start: `git clone` gives you the full dataset.

If you want to reproduce either curated subset from your own copy of the full public archive, or extend it with more reports, see `code/chapter_01/build_sample_archive.py` and `code/chapter_08/build_full_archive.py`.

4. Render the book

```
quarto render
```

Output is written to `_book/`. For live-editing a single chapter while you write:

```
quarto preview chapters/chapter_01.qmd
```

5. Working in Positron

Positron runs Quarto documents natively — open any `.qmd` file and use the **Render** button in the editor toolbar, or run individual code cells interactively (each ` ```{python} ` block behaves like a Jupyter cell). Recommended setup:

1. Open the `book/` folder as your Positron workspace root.
2. Select the `.venv` interpreter created in Step 2 via the interpreter picker (bottom status bar).
3. Use the **Quarto: Preview** command for live reload while editing a chapter.
4. Use the **Console** pane to run chapter scripts interactively (`python code/chapter_01/read_ddr.py ...`) — this is the fastest way to experiment with the code before committing it to a `.qmd`.

6. The companion pipeline (for Part II)

Chapters 6–12 reference **DDR_UTAH_FORGE**, a real, working DDR intelligence pipeline built specifically against this book's public archive — same architecture as the reference platform this project grew from (per-document extraction, hybrid retrieval, structured facts, causality analysis), adapted for Utah FORGE's real filenames and data. It's a separate repository — clone it alongside this book's repository, not inside it:

```
cd ..                        # up from book/ to your projects root
git clone https://github.com/djimrastephane/DDR_UTAH_FORGE.git
cd DDR_UTAH_FORGE
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Its own `data/` folder is gitignored — the repository is code only. Copy this book's `datasets/forge_archive/` into `DDR_UTAH_FORGE/data/raw/` (or point it at your own copy of the public Utah FORGE archive) and follow its `README.md` Quickstart to run the full pipeline and Streamlit dashboard.

> You don't need it to follow along
>
> Part II chapters are written so every technique works fully standalone: the simplified code in `code/chapter_06/` through `code/chapter_12/` in *this* book's repository requires no access to the companion pipeline. Where a chapter cites a specific file or a specific result from `DDR_UTAH_FORGE` (for example, the real 2,943-chunk global index in Chapter 8, or the real report-gap dates in Chapter 10), that result was independently verified against the pipeline's actual output before being written into the book, so you can trust the numbers even without running the pipeline yourself.

7. A note on data handling

This book's own data (`datasets/sample_ddrs/`, `datasets/forge_archive/`) is public and safe to commit, share, and publish — that's true throughout this repository. The moment you point this book's code at your own organisation's DDRs instead, that changes:

- Never commit real, confidential DDR PDFs, extracted text, or derived artefacts (embeddings, indexes) to a public repository.

- If you're experimenting on a shared machine, confirm your organisation's data classification policy covers running third-party Python packages (embedding models, OCR engines) against confidential well data.
- Add your own archive's path to `.gitignore` before dropping any confidential PDFs into a local `datasets/` folder for testing.

8. Troubleshooting

> Engineering Translation: PATH
>
> `PATH` is your computer's list of "which program to run by default when I type a command" — a lookup table, checked top to bottom, the first time you type something like `python`. Activating `.venv` (Step 2) temporarily moves this book's Python to the front of that list, so `python` means *this project's* Python until you close the terminal or deactivate it.

Symptom	Likely cause
`quarto render` fails on a code cell	Activate `.venv` before rendering — Quarto uses whichever Python is on `PATH`. Confirm with `which python` and `python -c "import pdfplumber"`.
`ModuleNotFoundError: sentence_transformers` (Chapter 4+)	Re-run `pip install -r requirements.txt`; this is a heavier dependency and sometimes needs a separate `pip install torch` first on some platforms.
FAISS import error on Apple Silicon (Chapter 8+)	Ensure you installed `faiss-cpu`, not `faiss-gpu` — the latter has no macOS wheel.
`read_ddr.py` reports no text extracted	Confirm you're pointing at a file in `datasets/sample_ddrs/` or `datasets/forge_archive/` — both are committed directly, so this shouldn't happen unless the path is wrong.

Appendix B: Oilfield Abbreviation Glossary

This glossary is a bigger version of the same equipment lookup table introduced in Chapter 2 — it supports Chapter 2's abbreviation expansion engine and its challenge exercise. It is broader than the `ABBREVIATIONS` dict built in Chapter 2 itself — that chapter's dictionary is deliberately scoped to terms verified present in this book's real Utah FORGE archive. This appendix adds enough additional common oilfield terms to give the expander real coverage on a first pass against a different archive. It is still not exhaustive — abbreviation usage varies by operator, rig contractor, and basin.

Terms marked **(FORGE)** are confirmed present in this book's real sample archive; the rest are common industry terms not found in this particular dataset but likely to appear in others.

Abbreviation	Expansion
AFE	authorization for expenditure (FORGE)
BHA	bottom hole assembly (FORGE)
BOP	blowout preventer
CBL	cement bond log
CIRC	circulate
COND	condition
CSG	casing
DFIT	diagnostic fracture injection test (FORGE)
DFS	days from spud (FORGE)
DHSV	downhole safety valve
DLS	dogleg severity (FORGE)
DOL	days on location (FORGE)
DP	drill pipe
ECD	equivalent circulating density (FORGE)
EMW	equivalent mud weight
FIT	formation integrity test
FR	from
GPM	gallons per minute (FORGE)
HWDP	heavy weight drill pipe (FORGE)
KOP	kick-off point
LOT	leak-off test
MD	measured depth (FORGE)
MIRU	move in, rig up
MW	mud weight
MWD	measurement while drilling (FORGE)
NMDC	non-magnetic drill collar (FORGE)
NPT	non-productive time (FORGE)
OBSD	observed

Abbreviation	Expansion
PBTD	plugged back total depth
PDC	polycrystalline diamond compact (bit type) (FORGE)
PJSM	pre-job safety meeting (FORGE)
POOH	pull out of hole
PPG	pounds per gallon
PU	pick up (pipe)
PUW	pick up weight
RIH	run in hole
RKB	rotary kelly bushing (FORGE)
ROP	rate of penetration (FORGE)
RPM	revolutions per minute (FORGE)
SICP	shut-in casing pressure
SIDPP	shut-in drill pipe pressure
SLM	slow (rate), or short for "slim hole" depending on context — verify against operator style guide
SPP	stand pipe pressure (FORGE)
STDS	stands (of drill pipe) (FORGE)
STK	stuck
TD	total depth
TIH	trip in hole
TOOH	trip out of hole
TVD	true vertical depth (FORGE)
UBHO	universal bottom hole orientation sub (FORGE)
UBI	ultrasonic borehole imager (wireline log) (FORGE)
WOB	weight on bit (FORGE)
WOC	wait on cement
WOW	wait on weather
XO	crossover (sub)

Abbreviations are not universal

The same three letters can mean different things across operators and basins — build your abbreviation dictionary from *your* archive's actual usage before trusting it against your own reports, and treat any ambiguous abbreviation (like `SLM` above) as a signal to check context rather than expand automatically.

References

Ji, Ziwei, Nayeon Lee, Rita Frieske, et al. 2023. "Survey of Hallucination in Natural Language Generation." In *ACM Computing Surveys*, No. 12, vol. 55.

Johnson, Jeff, Matthijs Douze, and Hervé Jégou. 2019. *Billion-Scale Similarity Search with GPUs*. https://arxiv.org/abs/1702.08734.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems* 33: 9459–74.

Matthes, Eric. 2023. *Python Crash Course: A Hands-on, Project-Based Introduction to Programming*. 3rd ed. No Starch Press.

Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.

Robertson, Stephen, and Hugo Zaragoza. 2009. "The Probabilistic Relevance Framework: BM25 and Beyond." *Foundations and Trends in Information Retrieval* 3: 333–89.

Society of Petroleum Engineers. 2022. "Digitalisation of Daily Drilling Reports: Challenges and Opportunities." *SPE Digital Transformation Series*.